Structure-from-Motion

Lecture 05 – Foundations of Spatial AI – detailed lecture notes

Notes based on the slides of L. Di Giammarino and E. Giacomini
Sapienza University of Rome – slide deck dated October 7, 2026

Abstract

These notes follow the slide deck section by section (slide ranges are given in every heading). Formal material – definitions, lemmas, theorems and proofs – is merged in at the point where the slides introduce the topic; it is taken from the standard references cited on the slides (Zhang 2000, Lowe 1999/2004, Hartley–Zisserman, Triggs et al. 2000, Schönberger–Frahm 2016, the SuperPoint and VGGT papers) and re-derived here. Every formal result is followed by a blue *Intuition* box. Boxes titled *Beyond the slides* mark facts that are not on the slides. Dashed boxes mark photos that should be copied in from the slide deck.

Contents

Agenda and Learning Objectives *(slides 1–2)* **2**

5.1 Camera Calibration *(slides 3–7)* **2**
5.1.1 What calibration is and how it is done *(slides 4–7)* . . . 2

5.2 Feature Detection and Description *(slides 8–20)* **7**
5.2.1 Point Features *(slides 9–11)* . . . 7
5.2.2 Scale Invariant Feature Transform (SIFT) *(slides 12–15)* . . . 8
5.2.3 Matching features *(slides 14–15)* . . . 9
5.2.4 SuperPoint: Learned Interest Points and Descriptors *(slides 16–20)* . . . 11

5.3 Two-Frame Structure-from-Motion *(slides 21–35)* **15**
5.3.1 Epipolar Geometry *(slides 22–23)* . . . 15
5.3.2 Structure of the Essential Matrix *(slides 24)* . . . 17
5.3.3 Estimating the Essential Matrix: the 8-point algorithm *(slides 25)* . . . 17
5.3.4 Decomposing the Essential Matrix *(slides 26)* . . . 18
5.3.5 Geometry of the Four Solutions *(slides 27)* . . . 19
5.3.6 Disambiguation by Cheirality *(slides 28)* . . . 20
5.3.7 Estimating the Epipolar Geometry with Unknown Intrinsics *(slides 29–30)* . . . 21
5.3.8 Triangulation *(slides 31–35)* . . . 22

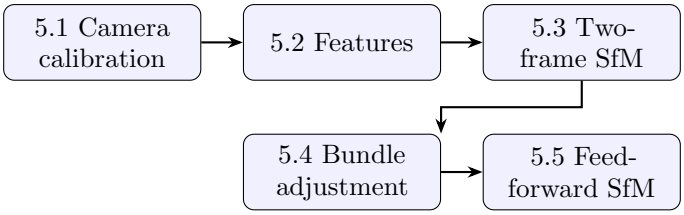
5.4 Bundle Adjustment *(slides 36–54)* **24**
5.4.1 Bundle Adjustment *(slides 37–39)* . . . 24
5.4.2 Challenges of Bundle Adjustment *(slides 40)* . . . 25
5.4.3 Bundle Adjustment as Nonlinear Least Squares *(slides 41)* . . . 25
5.4.4 Updates in the Tangent Space *(slides 42)* . . . 26
5.4.5 The Gauss–Newton Algorithm *(slides 43)* . . . 26
5.4.6 The Schur Complement *(slides 44)* . . . 27
5.4.7 Incremental Structure-from-Motion *(slides 45–51)* . . . 28
5.4.8 Building Rome in a Day, COLMAP SfM and COLMAP MVS *(slides 52–54)* . . . 31

5.5 Feed-Forward Structure-from-Motion *(slides 55–68)* **32**
5.5.1 Limits of Incremental Structure-from-Motion *(slides 56)* . . . 32
5.5.2 From Learned Components to Learned Reconstruction *(slides 57)* . . . 33
5.5.3 Visual Geometry Grounded Transformer (VGGT) *(slides 58)* . . . 34
5.5.4 Problem Definition *(slides 59)* . . . 34
5.5.5 Architecture: Alternating Attention *(slides 60)* . . . 35
5.5.6 Prediction Heads and Tracking *(slides 61)* . . . 36
5.5.7 Training Objective and Normalization *(slides 62)* . . . 36
5.5.8 Results: Camera Pose Estimation *(slides 63)* . . . 37
5.5.9 Results: Dense Geometry *(slides 64)* . . . 37
5.5.10 Results: Matching and Tracking *(slides 65)* . . . 38
5.5.11 Bundle Adjustment from VGGT Predictions *(slides 66)* . . . 39
5.5.12 Limitations *(slides 67)* . . . 39
5.5.13 Comparison of Incremental and Feed-Forward SfM *(slides 68)* . . . 40

Summary *(slide 69)* **40**

Agenda and Learning Objectives *(slides 1–2)*

The lecture takes us from a single calibrated camera to a full reconstruction pipeline and finally to networks that perform the whole reconstruction in one forward pass.



Learning objectives (slide 2).

1. Derive the epipolar constraint, estimate the essential matrix from correspondences, recover the relative pose up to scale, and triangulate a point from two views.
2. Formulate bundle adjustment as least squares over poses in SE(3) and 3D points, solve it with Gauss–Newton and the Schur complement, and describe the stages of incremental SfM.
3. Explain how a feed-forward transformer predicts cameras, depth, point maps and tracks, and how bundle adjustment refines them.

Notation used throughout.

Symbol	Meaning
$\bar{\boldsymbol{x}} = (x, y, 1)^\mathsf{T}$	homogeneous pixel coordinates
$\tilde{\boldsymbol{x}} = \boldsymbol{K}^{-1}\bar{\boldsymbol{x}}$	local ray direction of a pixel (calibrated camera)
$\boldsymbol{R}, \boldsymbol{t}$, $\boldsymbol{T} = [\boldsymbol{R}\|\boldsymbol{t}] \in \mathrm{SE}(3)$	rotation, translation, pose (world $\to$ camera)
$[\boldsymbol{t}]_\times$	skew-symmetric matrix with $[\boldsymbol{t}]_\times \boldsymbol{v} = \boldsymbol{t} \times \boldsymbol{v}$
$\tilde{\boldsymbol{E}} = [\boldsymbol{t}]_\times \boldsymbol{R}$, $\tilde{\boldsymbol{F}} = \boldsymbol{K}_2^{-\mathsf{T}} \tilde{\boldsymbol{E}} \boldsymbol{K}_1^{-1}$	essential / fundamental matrix
$\boldsymbol{x}_p^w$, $\boldsymbol{x}_{ip}^s$	3D world point p, its observation in image i
$\propto$	equality up to a nonzero scale factor

5.1 Camera Calibration *(slides 3–7)*

5.1.1 What calibration is and how it is done *(slides 4–7)*

Camera calibration is the process of finding the intrinsic and extrinsic parameters of a camera. Most commonly a *known calibration target* (an image or a checkerboard) is used.

Definition 5.1.1: Pinhole camera with intrinsics and extrinsics

A world point $\boldsymbol{x}^w \in \mathbb{R}^3$ is mapped to a pixel by

$$\lambda\, \bar{\boldsymbol{x}} = \boldsymbol{K}\,[\boldsymbol{R}\,|\,\boldsymbol{t}] \begin{pmatrix} \boldsymbol{x}^w \\ 1 \end{pmatrix}, \qquad \boldsymbol{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

where $\boldsymbol{K}$ holds the *intrinsics* (focal lengths f_x, f_y in pixels, skew s, principal point (c_x, c_y)) and $[\boldsymbol{R}|\boldsymbol{t}] \in \mathrm{SE}(3)$ are the *extrinsics*, i.e. the camera pose. Lens *distortion* (e.g. radial, $\boldsymbol{x}_d = \boldsymbol{x}\,(1 + k_1 r^2 + k_2 r^4)$ with $r = \|\boldsymbol{x}\|$ in normalized coordinates) is modelled by extra parameters applied before $\boldsymbol{K}$.

Intuition

$\boldsymbol{K}$ says how the metric ray through the pinhole is converted to pixels; $[\boldsymbol{R}|\boldsymbol{t}]$ says where the camera sits in the world. Calibration is an inverse problem: we look at something whose geometry we know perfectly (the target) and ask which camera would have produced those pixels.

The procedure on slides 4–7 has four stages (Fig. 1):

1. the known target is **captured in different poses** (slide 5);
2. **features** (e.g. corners) on the target are detected in the images (slide 6);
3. a **closed-form solution** initializes all parameters except the distortion parameters;
4. a **non-linear optimization** of all parameters minimizes the reprojection errors (slide 7), jointly over intrinsics and extrinsics (=poses).

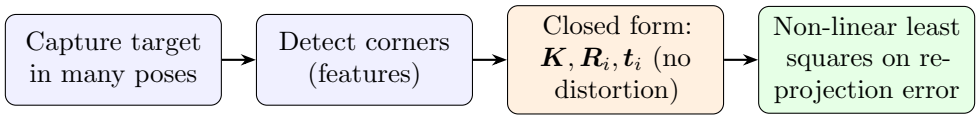


Figure 1: The calibration pipeline of Zhang (2000), redrawn from the description on slides 4–7.



Figure 2: A known calibration target held in front of several cameras (multi-camera calibration setup).

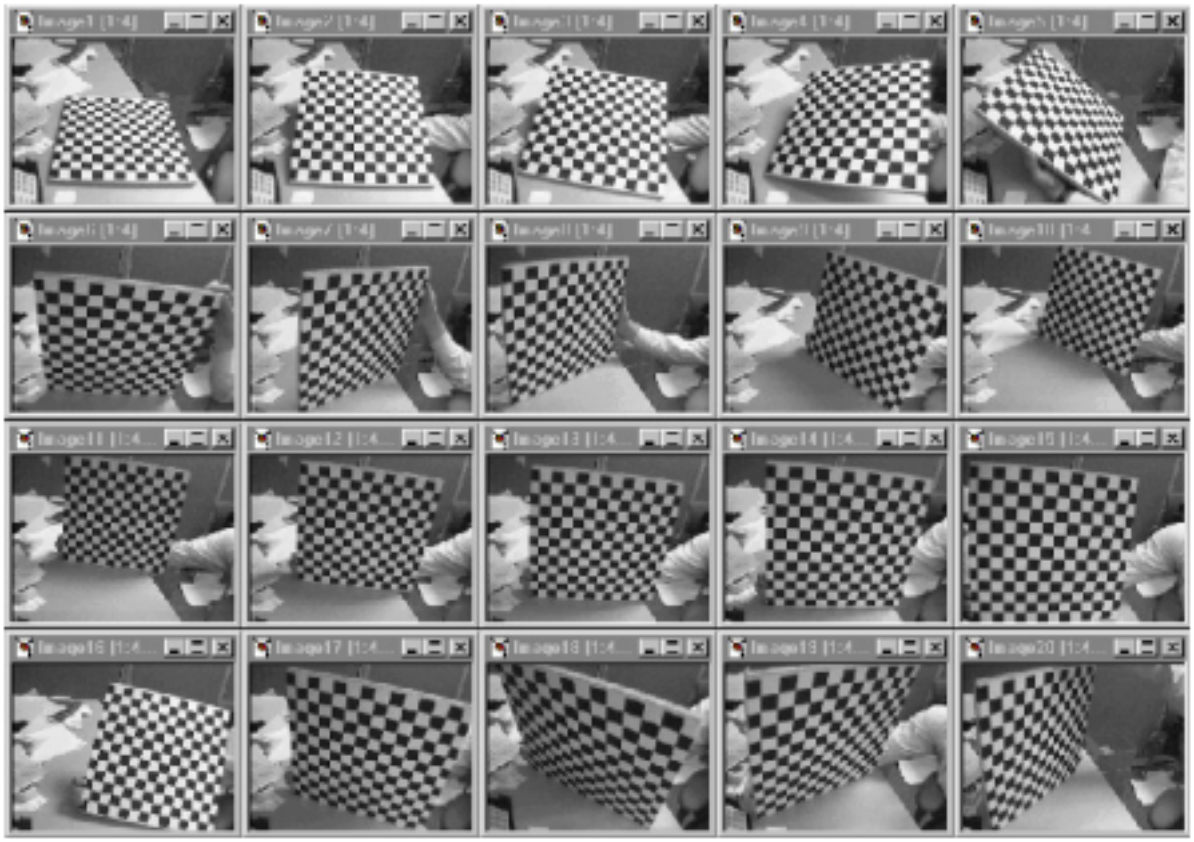


Figure 3: The calibration checkerboard captured in 20 different poses.

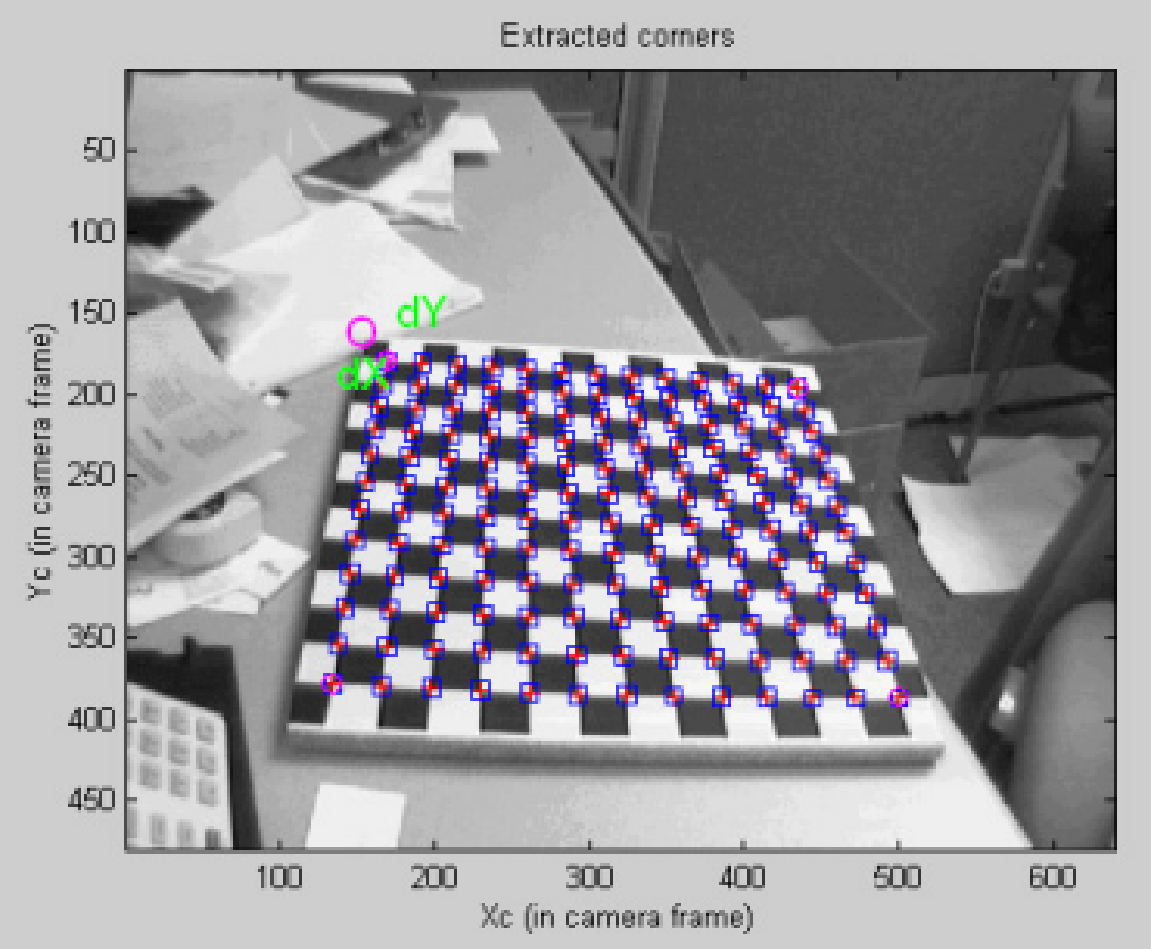


Figure 4: Corners detected on the target (red crosses, blue squares: extracted corners).

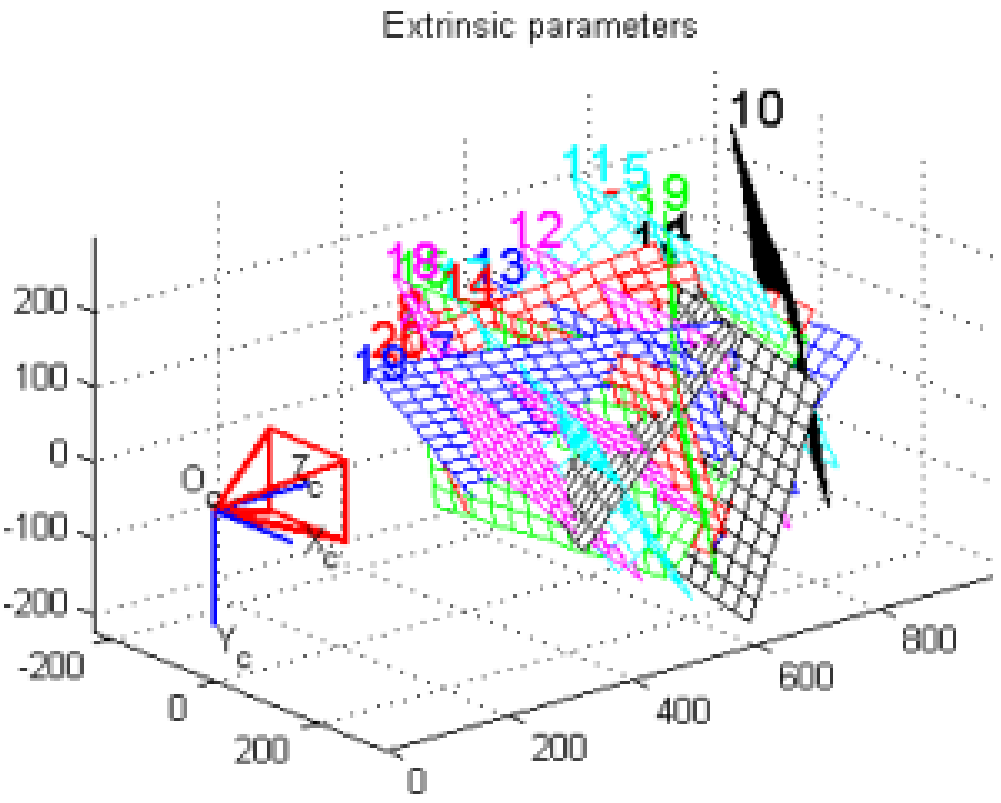


Figure 5: Recovered extrinsic parameters: the target planes in the camera frame after joint optimization (Zhang, TPAMI 2000).

Theorem 5.1.2: Zhang's constraints from a plane

Let the target lie in the plane $Z = 0$ of the world, so a target point is $(X, Y, 0)$. The homography $\boldsymbol{H} = [\boldsymbol{h}_1\ \boldsymbol{h}_2\ \boldsymbol{h}_3]$ mapping target coordinates $(X, Y, 1)^\mathsf{T}$ to pixels satisfies $\boldsymbol{H} = \lambda \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}]$. With $\boldsymbol{B} = \boldsymbol{K}^{-\mathsf{T}}\boldsymbol{K}^{-1}$ (symmetric, positive definite) every view yields the two linear equations in $\boldsymbol{B}$

$$\boldsymbol{h}_1^\mathsf{T}\boldsymbol{B}\,\boldsymbol{h}_2 = 0, \qquad \boldsymbol{h}_1^\mathsf{T}\boldsymbol{B}\,\boldsymbol{h}_1 = \boldsymbol{h}_2^\mathsf{T}\boldsymbol{B}\,\boldsymbol{h}_2.$$

Proof. For $Z = 0$ the column $\boldsymbol{r}_3$ is multiplied by zero, so $\lambda\bar{\boldsymbol{x}} = \boldsymbol{K}[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}](X, Y, 1)^\mathsf{T}$, which gives $\boldsymbol{H}$. From $\boldsymbol{h}_i = \lambda \boldsymbol{K}\boldsymbol{r}_i$ we get $\boldsymbol{r}_i = \lambda^{-1}\boldsymbol{K}^{-1}\boldsymbol{h}_i$ for $i = 1, 2$. Rotation columns are orthogonal and unit norm: $\boldsymbol{r}_1^\mathsf{T}\boldsymbol{r}_2 = 0$ gives $\boldsymbol{h}_1^\mathsf{T}\boldsymbol{K}^{-\mathsf{T}}\boldsymbol{K}^{-1}\boldsymbol{h}_2 = 0$ and $\boldsymbol{r}_1^\mathsf{T}\boldsymbol{r}_1 = \boldsymbol{r}_2^\mathsf{T}\boldsymbol{r}_2$ gives the second equation. □

Intuition

A rotation matrix has two "rules": its columns are perpendicular and equally long. A single view of a flat board gives a homography, and those two rules, pushed through the unknown $\boldsymbol{K}$, become two equations on $\boldsymbol{B}$ (which encodes $\boldsymbol{K}$). The board in one pose cannot tell you everything; a tilted board in a different pose tells you something new. That is why the target is captured in different poses.

Counting. $\boldsymbol{B}$ is symmetric, so it has 6 entries; it is defined up to scale, so 5 unknowns (matching the 5 intrinsics of $\boldsymbol{K}$). Each view gives 2 equations, hence ≥ 3 views for a general $\boldsymbol{K}$ (or 2 views if the skew $s = 0$ is assumed).

Algorithm 5.1.3: Zhang's calibration

1. For each view j estimate the homography $\boldsymbol{H}_j$ from the detected corners (DLT, as for homography estimation in lecture 01).
2. Stack the two equations of Thm. 5.1.2 for all views as $\boldsymbol{V}\boldsymbol{b} = 0$ and take the right singular vector of the smallest singular value; recover $\boldsymbol{K}$ from $\boldsymbol{b}$.
3. For every view: $\lambda = 1/\|\boldsymbol{K}^{-1}\boldsymbol{h}_1\|$, $\boldsymbol{r}_1 = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_1$, $\boldsymbol{r}_2 = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_2$, $\boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2$, $\boldsymbol{t} = \lambda\boldsymbol{K}^{-1}\boldsymbol{h}_3$. (Project $[\boldsymbol{r}_1\boldsymbol{r}_2\boldsymbol{r}_3]$ to the closest rotation with an SVD.) Distortion is initialized to zero.
4. Refine all parameters jointly by minimizing the reprojection error $\sum_{j,m}\|\bar{\boldsymbol{x}}_{jm} - \hat{\pi}(\boldsymbol{K}, \boldsymbol{k}, \boldsymbol{R}_j, \boldsymbol{t}_j, \boldsymbol{X}_m)\|^2$ with a non-linear least-squares solver (Levenberg–Marquardt).

Intuition

Steps 1–3 are cheap and linear but ignore noise statistics and distortion; they only have to be *good enough to start*. Step 4 is the honest estimator. This "closed form to initialize, non-linear least squares to polish" pattern returns again in bundle adjustment (Section 5.4).

Example 5.1.4: Size of a calibration problem (illustrative numbers)

A 9×6 checkerboard (54 inner corners) seen in 20 views gives $2 \cdot 54 \cdot 20 = 2160$ scalar residuals. The unknowns are 5 intrinsics, 2 distortion coefficients and $6 \cdot 20 = 120$ pose parameters: 127 in total. The problem is hugely over-determined, which is what makes it accurate.

5.2 Feature Detection and Description *(slides 8–20)*

5.2.1 Point Features *(slides 9–11)*

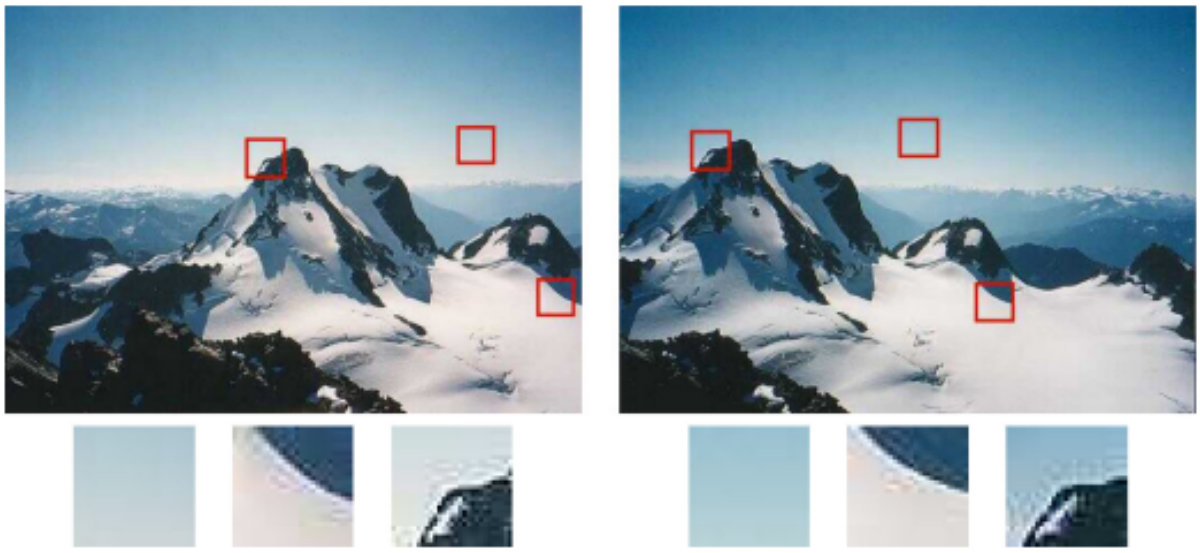


Figure 6: Two views of the same mountain scene; red boxes mark patches that are flat sky (uninformative), an edge, and a distinctive corner-like region (image credit: R. Szeliski).

A point feature is a pair $(\boldsymbol{k}, \boldsymbol{f})$: a *keypoint* $\boldsymbol{k} = (x, y)^\mathsf{T} \in \mathbb{R}^2$, the pixel location of a salient region, and a *descriptor* $\boldsymbol{f} \in \mathbb{R}^D$ that describes the appearance of the image around $\boldsymbol{k}$. An image yields a set of M features $\{(\boldsymbol{k}_j, \boldsymbol{f}_j)\}_{j=1}^{M}$. Two features $(\boldsymbol{k}, \boldsymbol{f})$ and $(\boldsymbol{k}', \boldsymbol{f}')$ in two images *match* when their descriptors are close, i.e. $\|\boldsymbol{f} - \boldsymbol{f}'\|$ is small.

Intuition

Think of a descriptor as a "fingerprint" of a patch. A flat piece of sky has a fingerprint that matches thousands of other patches (useless), whereas the tip of a mountain has a unique one. Point features describe local salient regions, they can be used to describe and match images from different viewpoints, and they form the basis of the sparse 3D reconstruction methods of this lecture.

The requirements (slide 11): features should be **invariant** to perspective effects and illumination, and the same 3D point should have **similar vectors independent of pose/viewpoint**. Plain RGB/intensity patches do not have this property, so we need something better.



Figure 7: Object recognition with local scale-invariant features: SIFT patches on three known products are found in a cluttered scene (Lowe, ICCV 1999).

5.2.2 Scale Invariant Feature Transform (SIFT) *(slides 12–15)*

Detection (slide 12). SIFT builds a *scale space* by iteratively filtering the image with a Gaussian. Adjacent scales are subtracted, yielding *Difference of Gaussian* (DoG) images; interest points (=blobs) are detected as extrema in the resulting scale space (Fig. 8).

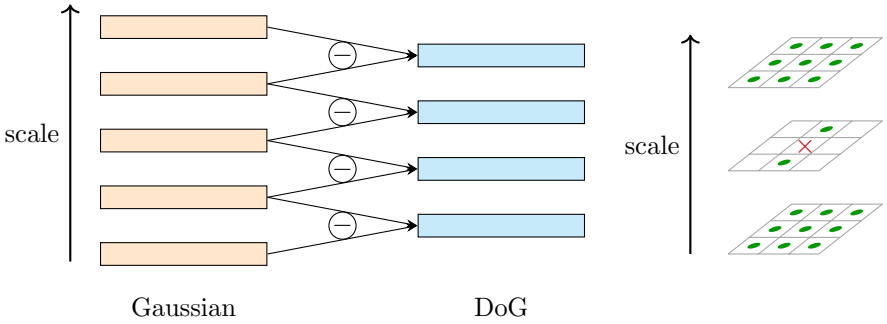


Figure 8: Left: first octave of the Gaussian scale space and the DoG images obtained by subtracting neighbouring scales. Right: a candidate (red cross) is kept only if it is larger or smaller than all its 26 neighbours in space and scale (green).

Proposition 5.2.2: DoG approximates the scale-normalized Laplacian

Let $G(\boldsymbol{x}, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\|\boldsymbol{x}\|^2/2\sigma^2}$. Then $\partial G/\partial\sigma = \sigma\nabla^2 G$ and, for a scale ratio k close to 1,

$$G(\boldsymbol{x}, k\sigma) - G(\boldsymbol{x}, \sigma) \;\approx\; (k-1)\,\sigma^2\nabla^2 G(\boldsymbol{x}, \sigma).$$

Proof. G solves the heat equation $\partial G/\partial(\sigma^2) = \frac{1}{2}\nabla^2 G$, i.e. $\partial G/\partial\sigma = \sigma\nabla^2 G$. A finite difference in σ gives $G(k\sigma) - G(\sigma) \approx (k\sigma - \sigma)\,\partial G/\partial\sigma = (k-1)\sigma^2\nabla^2 G$. □

Intuition

Subtracting two slightly different blurs is a cheap way to compute a blob detector (the Laplacian). Multiplying by σ^2 makes its response comparable across scales: a big blob at a coarse scale gives the same peak value as a small blob at a fine scale, so extrema over (position, scale) find blobs together with their size. That is where scale invariance comes from.

Description (slide 13). SIFT **rotates** the descriptor to **align with the dominant gradient orientation**; **gradient histograms** are computed for local sub-regions; all histograms are concatenated and normalized to form a **128D feature vector**.

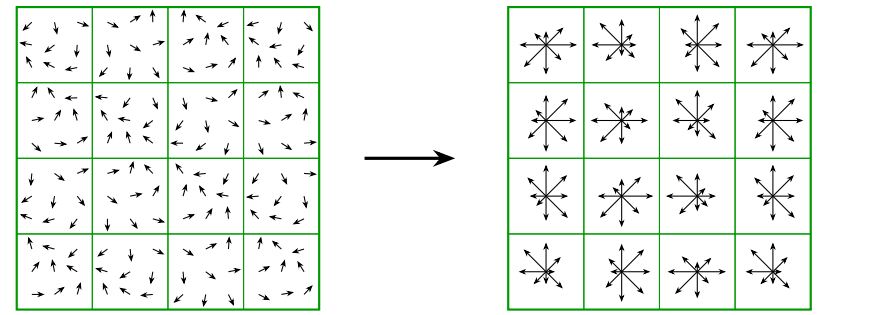


image gradients (4×4 cells) descriptor: $4 \times 4 \times 8 = 128$ numbers

Figure 9: Schematic of the SIFT descriptor: the gradients around the keypoint (after rotating to the dominant orientation) are pooled into an orientation histogram with 8 bins per cell; 4×4 cells give a 128D vector.

Intuition

Histograms of gradient *directions* throw away exact pixel positions inside each cell (robust to small shifts) but keep the coarse layout (distinctive). Rotating to the dominant orientation removes the camera's in-plane rotation; normalizing the vector removes affine illumination changes (gain).

Beyond the slides

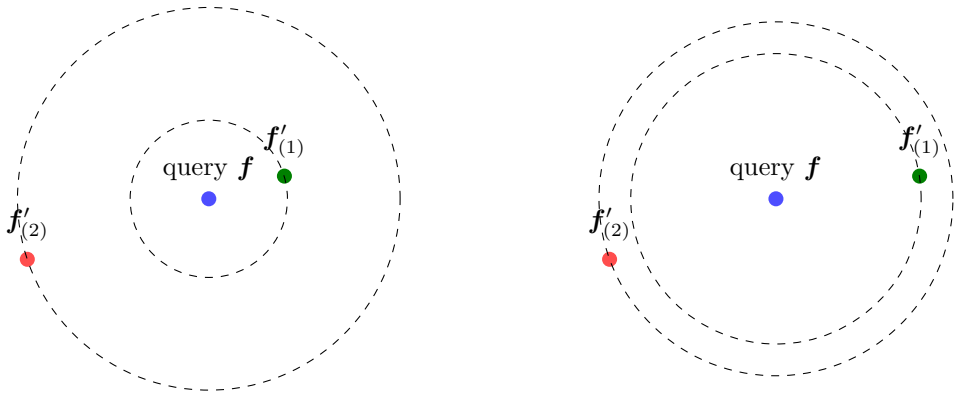
In Lowe's implementation, the vector is normalized, entries are clipped at 0.2, and it is re-normalized, which reduces the influence of non-linear illumination effects. The slide shows a 2×2 grid only for drawing convenience.

5.2.3 Matching features *(slides 14–15)*

Many algorithms exist (SIFT, SURF, U-SURF, BRISK, ORB, FAST and learned ones such as SuperPoint). SIFT was a seminal work thanks to its invariance and robustness and enabled large-scale SfM; despite being 20 years old it is still used, e.g. in COLMAP. Correspondences are retrieved by efficient nearest-neighbour search. Ambiguous matches are filtered with the **ratio test**: compute the distance to the closest neighbour divided by the distance to the second closest; a large ratio (> 0.8) indicates that the match might not be correct.

Definition 5.2.3: Lowe's ratio test

For a query descriptor $\boldsymbol{f}$ with nearest neighbours $\boldsymbol{f}'_{(1)}, \boldsymbol{f}'_{(2)}$ in the other image, accept the match $\boldsymbol{f} \leftrightarrow \boldsymbol{f}'_{(1)}$ iff $\dfrac{\|\boldsymbol{f} - \boldsymbol{f}'_{(1)}\|}{\|\boldsymbol{f} - \boldsymbol{f}'_{(2)}\|} < \tau$, with typically $\tau = 0.8$.



$d_1 = 1.04,\ d_2 = 2.53$, ratio 0.41: **accept** $d_1 = 1.92,\ d_2 = 2.34$, ratio 0.82: **reject**

Figure 10: Toy example in a 2D descriptor space. If the best match is not clearly better than the runner-up, the match is ambiguous.

Intuition

If I ask "who is my best friend?" and the answer is "person A, and person B is almost as close", I should not trust the answer. The ratio test measures that gap. With ratio 0.41 the closest descriptor is more than twice as close as the next; with ratio 0.82 it is basically a coin flip between two candidates (e.g. a repeated window on a facade).

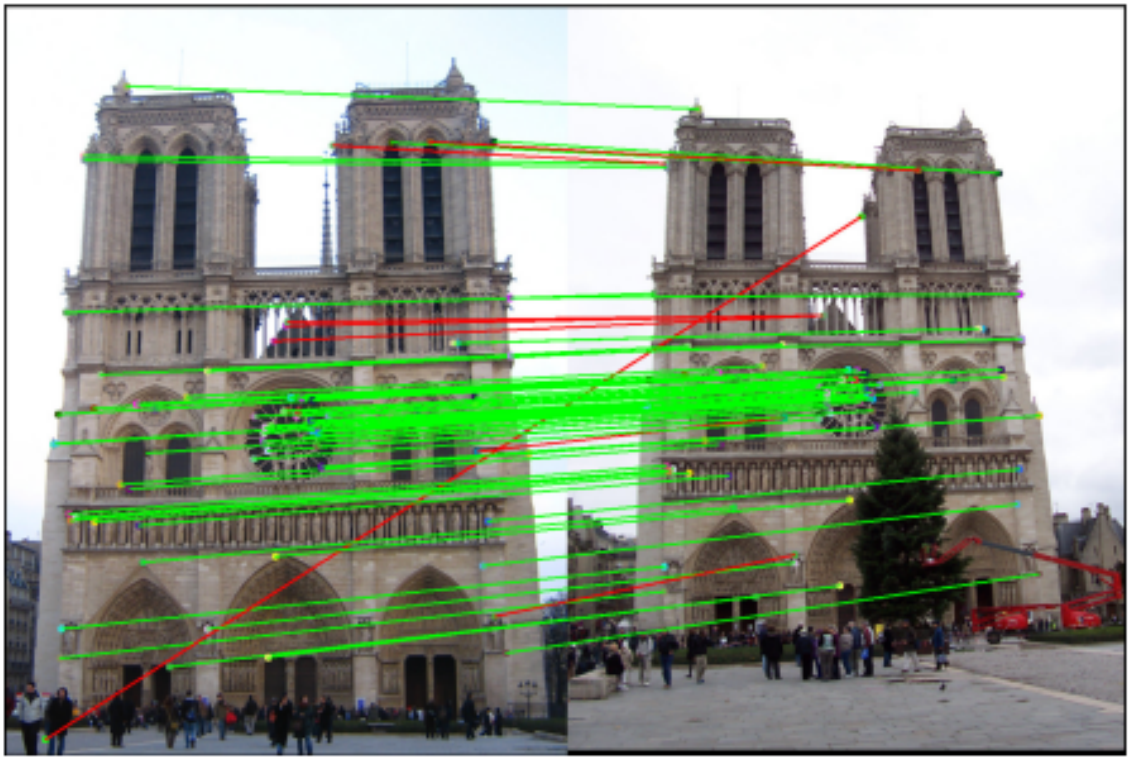


Figure 11: Two images of Notre-Dame with SIFT correspondences (green: correct, red: wrong) (image credits: J. Hays).

5.2.4 SuperPoint: Learned Interest Points and Descriptors *(slides 16–20)*

Idea (slide 16). A fully convolutional network computes interest points and their descriptors in one forward pass over the full image. Training is **self-supervised** in three stages (Fig. 12): (a) a base detector, MagicPoint, is trained on rendered shapes whose corners are known; (b) it labels real, unlabeled images by Homographic Adaptation; (c) detector and descriptor are trained jointly on image pairs related by a known homography. Runtime: about 13 ms for a 480×640 image on one Titan X GPU (about 70 fps).

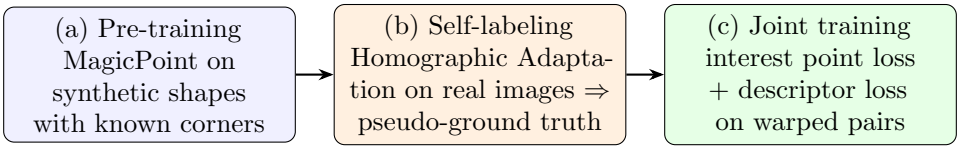


Figure 12: The three SuperPoint training stages (slide 16).

Base detector and training details (slide 17). MagicPoint is trained for 200,000 iterations on *Synthetic Shapes* (quads, triangles, cubes, stars, lines, checkerboards, quad grids), rendered on the fly so that no image is seen twice. Pseudo-ground truth: 80,000 MS-COCO images, grayscale, 240×320, Homographic Adaptation with $N_h = 100$, run twice. Joint training: one random homography per image (milder than in adaptation), cross-entropy for interest points, a hinge loss for descriptors. Optimization: batch size 32, Adam with learning rate 10^{-3}, augmentation (Gaussian noise, motion blur, brightness), descriptor length $D = 256$.

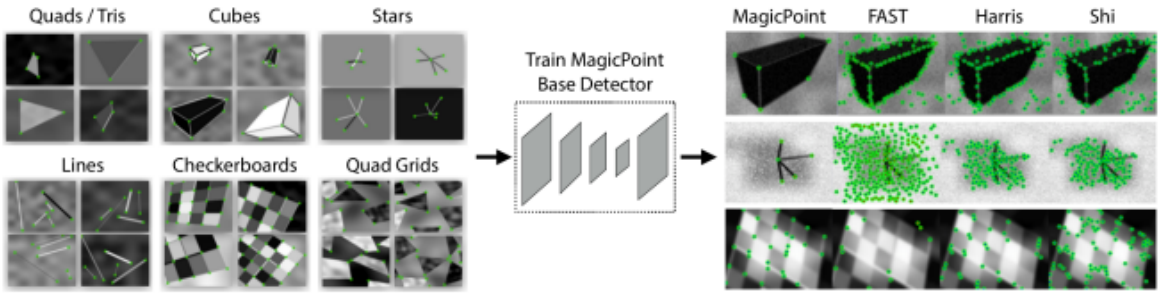


Figure 13: Left: Synthetic Shapes with known corners. Right: MagicPoint against FAST, Harris and Shi–Tomasi corners on noisy images.

Homographic Adaptation (slide 18).

Definition 5.2.4: Covariant detector and Homographic Adaptation

Let f_θ map an image to a heatmap of interest points and let $\tilde{\boldsymbol{H}}(\cdot)$ denote warping an image or heatmap by the homography $\tilde{\boldsymbol{H}}$. f_θ is *covariant* if $f_\theta(\tilde{\boldsymbol{H}}(\boldsymbol{I})) = \tilde{\boldsymbol{H}}(f_\theta(\boldsymbol{I}))$. Homographic Adaptation computes the label heatmap

$$\hat{F}(\boldsymbol{I}) = \frac{1}{N_h} \sum_{i=1}^{N_h} \tilde{\boldsymbol{H}}_i^{-1}\big(f_\theta(\tilde{\boldsymbol{H}}_i(\boldsymbol{I}))\big),$$

with random homographies $\tilde{\boldsymbol{H}}_i$ and $N_h = 100$ warps.

Lemma 5.2.5: Averaging over warps

If f_θ is exactly covariant, then $\hat{F}(\boldsymbol{I}) = f_\theta(\boldsymbol{I})$ for every N_h.

Proof. Each term equals $\tilde{\boldsymbol{H}}_i^{-1}\big(\tilde{\boldsymbol{H}}_i(f_\theta(\boldsymbol{I}))\big) = f_\theta(\boldsymbol{I})$, and the mean of N_h equal terms is that term. □

Intuition

A real network is only approximately covariant: it finds a corner from one viewpoint and misses it from another. Looking at the image through 100 different warps and voting (after un-warping the heatmaps) keeps the points the detector finds *consistently* and averages out accidental misses, so the labels impose covariance on the training data. The labels are generated on everyday photographs with no annotation of interest points: that is what "self-supervised" means here.

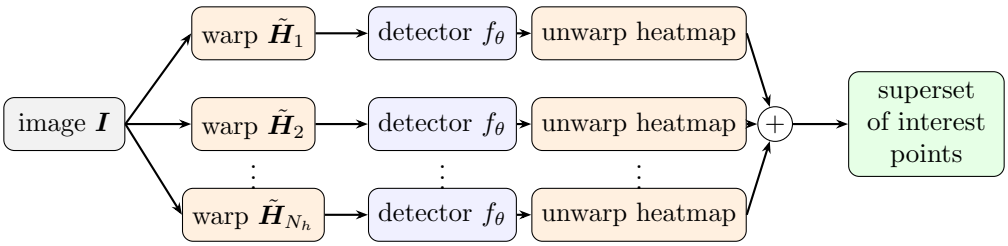


Figure 14: Homographic Adaptation (redrawn from slide 18): detect on warped copies, un-warp, average, and extract the pseudo-ground-truth points.

Architecture (slide 19). The encoder consists of eight 3×3 convolutions and three 2×2 max pools, hence one cell c per 8×8 pixels. Two heads share it (Fig. 15).

- **Keypoint head:** $\boldsymbol{S} \in \mathbb{R}^{H/8 \times W/8 \times 65}$. Each cell is classified into 65 classes: one per pixel and one for "no keypoint" (the 65th class). The loss is the cross-entropy against the pseudo-ground-truth label y_c, averaged over the $HW/64$ cells:

$$\mathcal{L}_p = -\frac{64}{HW} \sum_c \log \frac{\exp \boldsymbol{S}[c, y_c]}{\sum_{j=1}^{65} \exp \boldsymbol{S}[c, j]}.$$

- **Descriptor head:** $\boldsymbol{F} \in \mathbb{R}^{H/8 \times W/8 \times 256}$, one unit-length descriptor $\boldsymbol{f} \in \mathbb{R}^{256}$ per cell, interpolated at each keypoint $\boldsymbol{k}$ (bi-cubic interpolation, L2 normalization).
- **Descriptor loss:** over all pairs of cells of the two images, with $s = 1$ if the homography maps one onto the other,

$$s \max(0,\, 1 - \boldsymbol{f}^\top \boldsymbol{f}') + (1 - s) \max(0,\, \boldsymbol{f}^\top \boldsymbol{f}' - 0.2).$$

Self-supervision through known homography.

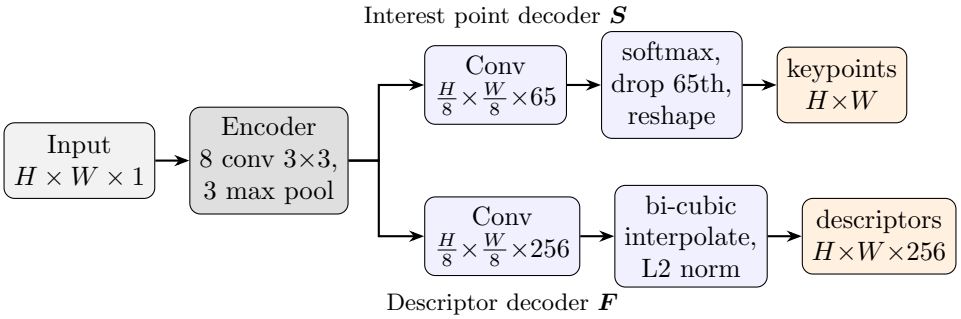


Figure 15: SuperPoint architecture (redrawn from slide 19).

Example 5.2.6: Tensor sizes for a 480×640 image

There are $60 \times 80 = 4800$ cells. The keypoint head outputs a $60 \times 80 \times 65$ tensor; after the softmax the dustbin channel is dropped and the remaining 64 channels are reshaped to 8×8 pixels, giving a full-resolution 480×640 heatmap. The descriptor head outputs $60 \times 80 \times 256$, i.e. a coarse semi-dense grid of descriptors that is interpolated at the detected keypoints. At 13 ms per image this is ≈ 77 frames per second (the slide quotes 70).

Intuition

Instead of asking "is this pixel a corner?" 480×640 times, the network asks each 8×8 cell "which of your 64 pixels is the corner, or is there none?". That is a 65-way classification, solved with an ordinary softmax and cross-entropy. The descriptor loss pulls descriptors of cells that the homography matches towards the same direction (dot product close to 1, margin 1) and pushes non-matching ones apart (dot product below 0.2).

Results on HPatches (slide 20). HPatches has 116 scenes, 57 with large illumination changes and 59 with large viewpoint changes; at most 1000 points per method at 480×640.

Method	ε=1	ε=3	ε=5	Rep.	MLE↓	NN mAP	M. Score
SuperPoint	0.310	**0.684**	**0.829**	0.581	1.158	**0.821**	**0.470**
LIFT	0.284	0.598	0.717	0.449	1.102	0.664	0.315
SIFT	**0.424**	0.676	0.759	0.495	**0.833**	0.694	0.313
ORB	0.150	0.395	0.538	**0.641**	1.157	0.735	0.266

Table 1: HPatches. ε=1, 3, 5: share of image pairs whose estimated homography is correct within ε pixels; Rep.: detector repeatability; MLE: mean localization error in pixels (lower is better); NN mAP: mean average precision of nearest-neighbour descriptor matching; M. Score: matching score.

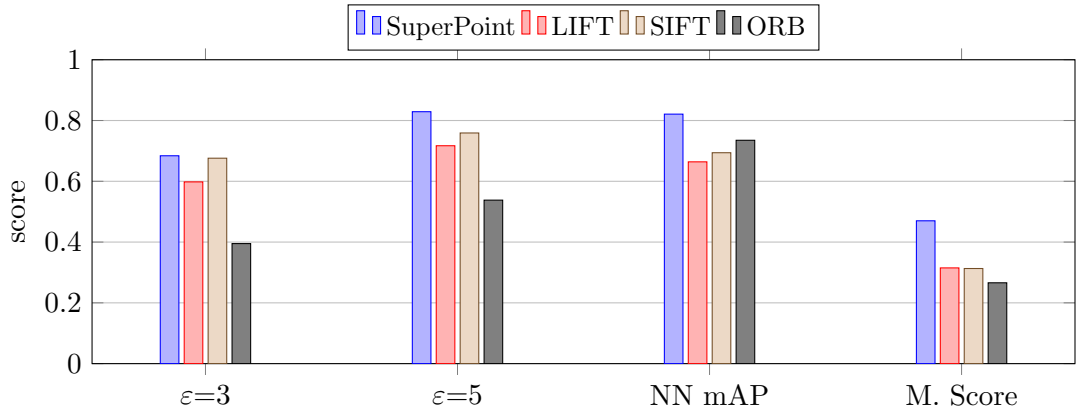


Figure 16: Selected HPatches metrics from the table (higher is better).

Intuition

SuperPoint wins on the metrics that depend on *descriptors* and on how well matches are spread across the image (homography accuracy, NN mAP, matching score), while SIFT is still best at sub-pixel *localization* (lowest MLE) and at ε=1. On slide 20 the fourth row shows a failure of SuperPoint and LIFT under an extreme in-plane rotation, a case absent from their training data: a learned method is only as invariant as its training augmentations.

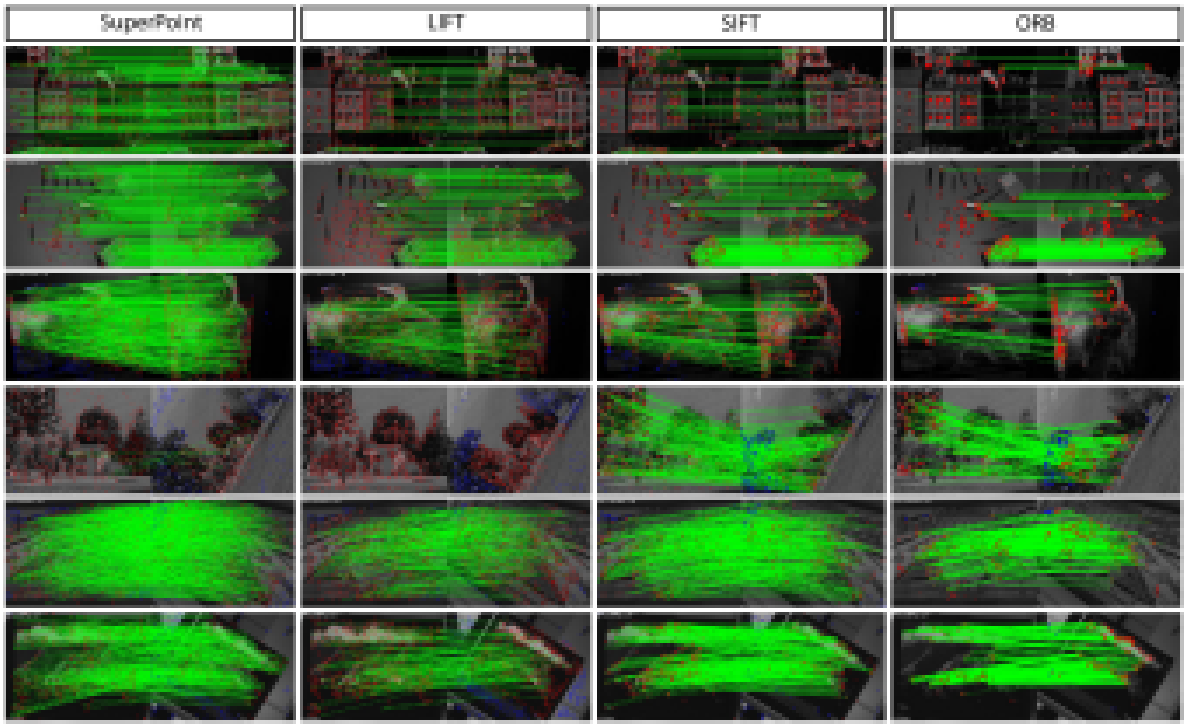


Figure 17: Qualitative matches of SuperPoint, LIFT, SIFT and ORB on six HPatches pairs (green: matches; the fourth row shows an extreme in-plane rotation).

5.3 Two-Frame Structure-from-Motion *(slides 21–35)*

5.3.1 Epipolar Geometry *(slides 22–23)*

Goal: recovery of camera pose (and 3D structure) from image correspondences. The required relationships are described by the two-view *epipolar geometry*.

- Let $\boldsymbol{R}$ and $\boldsymbol{t}$ denote the relative pose between two perspective cameras.
- A 3D point $\boldsymbol{x}$ is projected to pixel $\bar{\boldsymbol{x}}_1$ in image 1 and to pixel $\bar{\boldsymbol{x}}_2$ in image 2.
- The 3D point $\boldsymbol{x}$ and the two camera centers span the **epipolar plane**.
- The correspondence of $\bar{\boldsymbol{x}}_1$ in image 2 must lie on the **epipolar line** $\tilde{\boldsymbol{l}}_2$ in image 2.
- All epipolar lines pass through the **epipole**.

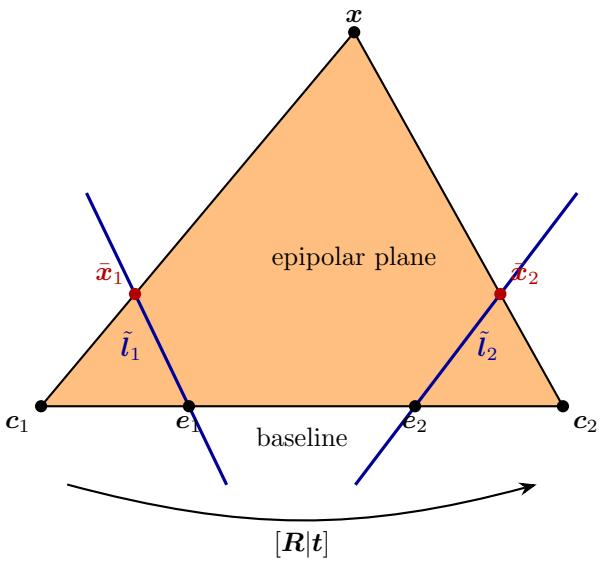


Figure 18: Epipolar geometry (redrawn from slide 22). The image planes are drawn edge-on: their intersection with the epipolar plane is the epipolar line.

Definition 5.3.1: Epipolar plane, lines and epipoles

For a world point $\boldsymbol{x}$ and camera centers $\boldsymbol{c}_1, \boldsymbol{c}_2$, the *epipolar plane* is the plane through $\boldsymbol{x}, \boldsymbol{c}_1, \boldsymbol{c}_2$. The *baseline* is the line $\boldsymbol{c}_1\boldsymbol{c}_2$. The *epipole* $\boldsymbol{e}_i$ in image i is the image of the other camera's center. The *epipolar line* $\tilde{\boldsymbol{l}}_2$ of a pixel $\bar{\boldsymbol{x}}_1$ is the intersection of the epipolar plane with image 2.

Intuition

If I see a point in camera 1, I know it lies somewhere on the ray through $\boldsymbol{c}_1$ and $\bar{\boldsymbol{x}}_1$, but not how far. Camera 2 sees that whole ray as a *line*. So the 2D search for a match shrinks to a 1D search along the epipolar line (slide 30 shows this on real images). Because every ray passes through $\boldsymbol{c}_1$, all the epipolar lines in image 2 pass through the projection of $\boldsymbol{c}_1$: the epipole.

Derivation of the epipolar constraint (slide 23). Let $\boldsymbol{K}_i$ be the calibration matrix of camera i, and $\tilde{\boldsymbol{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$ the local ray direction of pixel $\bar{\boldsymbol{x}}_i$ in camera i.

Theorem 5.3.2: Epipolar constraint

Let $\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$ relate the coordinates of a 3D point in the two camera frames, and let $\tilde{\boldsymbol{E}} = [\boldsymbol{t}]_\times \boldsymbol{R}$ be the *essential matrix*. Then

$$\tilde{\boldsymbol{x}}_2^\mathsf{T}\, \tilde{\boldsymbol{E}}\, \tilde{\boldsymbol{x}}_1 = 0.$$

Proof. A point's camera coordinates are proportional to its ray direction: $\boldsymbol{x}_i \propto \tilde{\boldsymbol{x}}_i$, say $\boldsymbol{x}_1 = z_1\tilde{\boldsymbol{x}}_1$, $\boldsymbol{x}_2 = z_2\tilde{\boldsymbol{x}}_2$. Then $z_2\tilde{\boldsymbol{x}}_2 = z_1\boldsymbol{R}\tilde{\boldsymbol{x}}_1 + \boldsymbol{t}$, so $\tilde{\boldsymbol{x}}_2 \propto \boldsymbol{R}\tilde{\boldsymbol{x}}_1 + s\,\boldsymbol{t}$ for some scalar s (slide: $\boldsymbol{x}_2 \propto \boldsymbol{R}\tilde{\boldsymbol{x}}_1 + s\boldsymbol{t}$). Taking the cross product with $\boldsymbol{t}$ (i.e. multiplying by $[\boldsymbol{t}]_\times$, which kills $\boldsymbol{t}$) gives $[\boldsymbol{t}]_\times\tilde{\boldsymbol{x}}_2 \propto [\boldsymbol{t}]_\times\boldsymbol{R}\tilde{\boldsymbol{x}}_1$. Taking the dot product with $\tilde{\boldsymbol{x}}_2$: the left side is $\tilde{\boldsymbol{x}}_2^\mathsf{T}[\boldsymbol{t}]_\times\tilde{\boldsymbol{x}}_2 = 0$ because $[\boldsymbol{t}]_\times$ is skew-symmetric ($\boldsymbol{v}^\mathsf{T}[\boldsymbol{t}]_\times\boldsymbol{v} = \boldsymbol{v}\cdot(\boldsymbol{t}\times\boldsymbol{v}) = 0$ by the triple product). Hence $\tilde{\boldsymbol{x}}_2^\mathsf{T}[\boldsymbol{t}]_\times\boldsymbol{R}\tilde{\boldsymbol{x}}_1 = 0$. □

Intuition

The three vectors $\tilde{\boldsymbol{x}}_2$, $\boldsymbol{R}\tilde{\boldsymbol{x}}_1$ and $\boldsymbol{t}$ all lie in the epipolar plane, so they are linearly dependent; their scalar triple product, $\tilde{\boldsymbol{x}}_2\cdot(\boldsymbol{t}\times\boldsymbol{R}\tilde{\boldsymbol{x}}_1)$, vanishes. That is the entire constraint, written as a bilinear form $\tilde{\boldsymbol{x}}_2^\mathsf{T}\tilde{\boldsymbol{E}}\,\tilde{\boldsymbol{x}}_1 = 0$. It does not involve depth: we can test correspondences before knowing any 3D structure.

Example 5.3.3: The rectified stereo pair

Take $\boldsymbol{R} = \boldsymbol{I}$ and $\boldsymbol{t} = (1,0,0)^\mathsf{T}$ (a pure sideways shift of one unit). Then $\tilde{\boldsymbol{E}} = [\boldsymbol{t}]_\times = \left(\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{smallmatrix}\right)$ and for $\tilde{\boldsymbol{x}}_i = (u_i, v_i, 1)^\mathsf{T}$: $\tilde{\boldsymbol{E}}\tilde{\boldsymbol{x}}_1 = (0,-1,v_1)^\mathsf{T}$, so $\tilde{\boldsymbol{x}}_2^\mathsf{T}\tilde{\boldsymbol{E}}\tilde{\boldsymbol{x}}_1 = v_1 - v_2 = 0$: corresponding points have the **same row**, i.e. epipolar lines are horizontal. This is the stereo setting of lecture 04. With $\boldsymbol{X} = (X,Y,Z)^\mathsf{T}$ in camera 1, $u_1 = X/Z$ and $u_2 = (X+1)/Z$, so $u_2 - u_1 = 1/Z$. If $u_1 = 0.1$ and $u_2 = 0.3$, then $Z = 5$ (in units of the baseline).

5.3.2 Structure of the Essential Matrix *(slides 24)*

Proposition 5.3.4: Properties of $\tilde{E}$

Let $\tilde{\boldsymbol{E}} = [\boldsymbol{t}]_\times \boldsymbol{R}$ with $\boldsymbol{R} \in \mathrm{SO}(3)$ and $\boldsymbol{t} \neq 0$. Then

(i) $\tilde{\boldsymbol{E}}\tilde{\boldsymbol{E}}^\mathsf{T} = [\boldsymbol{t}]_\times [\boldsymbol{t}]_\times^\mathsf{T} = \|\boldsymbol{t}\|^2 \boldsymbol{I} - \boldsymbol{t}\boldsymbol{t}^\mathsf{T}$, hence the singular values of $\tilde{\boldsymbol{E}}$ are $(\|\boldsymbol{t}\|, \|\boldsymbol{t}\|, 0)$ and $\tilde{\boldsymbol{E}}$ has rank 2;

(ii) $\boldsymbol{t}^\mathsf{T}\tilde{\boldsymbol{E}} = 0$ (left null vector $\boldsymbol{t} \propto \boldsymbol{e}_2$) and $\tilde{\boldsymbol{E}}\,\boldsymbol{R}^\mathsf{T}\boldsymbol{t} = 0$ (right null vector $\boldsymbol{R}^\mathsf{T}\boldsymbol{t} \propto \boldsymbol{e}_1$): each epipole is the image of the other camera's center;

(iii) $\tilde{\boldsymbol{E}}$ has 5 degrees of freedom: 3 for $\boldsymbol{R}$ and 3 for $\boldsymbol{t}$, minus one for the scale;

(iv) (scale ambiguity) scaling the scene and $\boldsymbol{t}$ by the same α leaves every image point unchanged, since $\alpha\boldsymbol{x}_2 = \boldsymbol{R}(\alpha\boldsymbol{x}_1) + \alpha\boldsymbol{t}$. Hence $\|\boldsymbol{t}\|$ cannot be observed and $\tilde{\boldsymbol{E}}$ is defined only up to scale, sign included.

Proof. (i) $\tilde{\boldsymbol{E}}\tilde{\boldsymbol{E}}^\mathsf{T} = [\boldsymbol{t}]_\times \boldsymbol{R}\boldsymbol{R}^\mathsf{T}[\boldsymbol{t}]_\times^\mathsf{T} = [\boldsymbol{t}]_\times[\boldsymbol{t}]_\times^\mathsf{T} = \|\boldsymbol{t}\|^2\boldsymbol{I} - \boldsymbol{t}\boldsymbol{t}^\mathsf{T}$, which has eigenvalues $\|\boldsymbol{t}\|^2$ (twice, on $\boldsymbol{t}^\perp$) and 0 (along $\boldsymbol{t}$). (ii) $\boldsymbol{t}^\mathsf{T}[\boldsymbol{t}]_\times = -([\boldsymbol{t}]_\times\boldsymbol{t})^\mathsf{T} = 0$, and $\tilde{\boldsymbol{E}}\boldsymbol{R}^\mathsf{T}\boldsymbol{t} = [\boldsymbol{t}]_\times\boldsymbol{t} = 0$. The camera-1 center in camera-2 coordinates is $\boldsymbol{R}\cdot\boldsymbol{0} + \boldsymbol{t} = \boldsymbol{t}$, so $\boldsymbol{e}_2 \propto \boldsymbol{t}$; the camera-2 center in camera-1 coordinates is $-\boldsymbol{R}^\mathsf{T}\boldsymbol{t}$, so $\boldsymbol{e}_1 \propto \boldsymbol{R}^\mathsf{T}\boldsymbol{t}$. (iii) and (iv) follow from counting and from the displayed identity. □

Intuition

Rank 2 is the algebraic signature of "all epipolar lines pass through one point": the epipole is the null direction. The scale ambiguity is the same effect that makes a photo of a model train indistinguishable from a photo of a real train: a single moving camera can recover the *shape* of the scene and its *motion direction*, never the absolute size, unless something metric (a known length, an IMU, a stereo baseline) is available.

Theorem 5.3.5: Characterization of essential matrices

A real 3×3 matrix is an essential matrix (up to scale) iff its SVD has the form $\boldsymbol{U}\,\mathrm{diag}(\sigma,\sigma,0)\boldsymbol{V}^\mathsf{T}$ with $\sigma > 0$.

Proof. ($\Rightarrow$) is Prop. 5.3.4(i). ($\Leftarrow$) Normalize $\sigma = 1$ and choose $\boldsymbol{U}, \boldsymbol{V}$ with $\det = +1$ (flip the sign of the third column if needed: this does not change the matrix because $\sigma_3 = 0$). Theorem 5.3.8 below shows $\boldsymbol{U}\,\mathrm{diag}(1,1,0)\boldsymbol{V}^\mathsf{T} = [\boldsymbol{u}_3]_\times\boldsymbol{R}$ with $\boldsymbol{R} = \boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T} \in \mathrm{SO}(3)$. □

5.3.3 Estimating the Essential Matrix: the 8-point algorithm *(slides 25)*

Each correspondence $\tilde{\boldsymbol{x}}_1 = (x_1, y_1, 1)^\mathsf{T}$, $\tilde{\boldsymbol{x}}_2 = (x_2, y_2, 1)^\mathsf{T}$ gives one *linear* equation in the nine entries of $\tilde{\boldsymbol{E}}$. Stacking the rows of a matrix into a vector,

$$\tilde{\boldsymbol{x}}_2^\mathsf{T}\tilde{\boldsymbol{E}}\tilde{\boldsymbol{x}}_1 = \mathrm{vec}\big(\tilde{\boldsymbol{x}}_2^\mathsf{T}\tilde{\boldsymbol{E}}\tilde{\boldsymbol{x}}_1\big) = \big(\tilde{\boldsymbol{x}}_1 \otimes \tilde{\boldsymbol{x}}_2\big)^\mathsf{T}\,\mathrm{vec}(\tilde{\boldsymbol{E}}) = \boldsymbol{a}^\mathsf{T}\,\mathrm{vec}(\tilde{\boldsymbol{E}}) = 0,$$

$$\boldsymbol{a} = (x_2x_1,\ x_2y_1,\ x_2,\ y_2x_1,\ y_2y_1,\ y_2,\ x_1,\ y_1,\ 1)^\mathsf{T}, \qquad \mathrm{vec}(\tilde{\boldsymbol{E}}) = (e_{11}, e_{12}, \ldots, e_{33})^\mathsf{T}.$$

Algorithm 5.3.6: Normalized 8-point algorithm

1. (Normalization) Whiten the observations to zero mean and unit variance, because products of two measurements amplify noise unevenly (Hartley 1997).
2. Stack the rows $\boldsymbol{a}_i^\mathsf{T}$ of $N \geq 8$ correspondences into $\boldsymbol{A} \in \mathbb{R}^{N\times 9}$ and solve $\min\|\boldsymbol{A}\,\mathrm{vec}(\tilde{\boldsymbol{E}})\|$ s.t. $\|\mathrm{vec}(\tilde{\boldsymbol{E}})\| = 1$: the solution is the last right singular vector of $\boldsymbol{A}$.
3. (Projection) The estimate is not an essential matrix when there is noise. Take its SVD $\boldsymbol{U}\,\mathrm{diag}(\sigma_1,\sigma_2,\sigma_3)\boldsymbol{V}^\mathsf{T}$; the closest essential matrix in the Frobenius norm has singular values $(\bar{\sigma},\bar{\sigma},0)$ with $\bar{\sigma} = (\sigma_1+\sigma_2)/2$. As the scale is free, take $\tilde{\boldsymbol{E}} = \boldsymbol{U}\,\mathrm{diag}(1,1,0)\boldsymbol{V}^\mathsf{T}$.
4. (Undo the normalization.)

Intuition

Eight equations for nine unknowns defined up to scale: that is the minimal linear count. The $\boldsymbol{A}$-matrix is built so that $\boldsymbol{A}\,\mathrm{vec}(\tilde{\boldsymbol{E}})$ lists the "epipolar errors" of all correspondences, and the last singular vector is the $\tilde{\boldsymbol{E}}$ that makes their sum of squares smallest. Step 3 then repairs the answer: the noisy $\tilde{\boldsymbol{E}}$ has three unequal singular values, so we average the top two and zero the third (which restores rank 2 and equal non-zero singular values).

Lemma 5.3.7: Projection onto essential matrices

Let $\boldsymbol{M} = \boldsymbol{U}\,\mathrm{diag}(\sigma_1,\sigma_2,\sigma_3)\boldsymbol{V}^\mathsf{T}$ with $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$. Among matrices of the form $\boldsymbol{U}'\,\mathrm{diag}(\sigma,\sigma,0)\boldsymbol{V}'^\mathsf{T}$ the one closest to $\boldsymbol{M}$ in the Frobenius norm is $\boldsymbol{U}\,\mathrm{diag}(\bar{\sigma},\bar{\sigma},0)\boldsymbol{V}^\mathsf{T}$ with $\bar{\sigma} = (\sigma_1+\sigma_2)/2$.

Sketch. The Frobenius distance between matrices with singular values $\boldsymbol{\sigma}$ and $\boldsymbol{\sigma}'$ (sorted) is minimized by sharing the singular vectors and is then $\|\boldsymbol{\sigma}-\boldsymbol{\sigma}'\|_2$ (Mirsky/von Neumann). Minimizing $(\sigma_1-s)^2+(\sigma_2-s)^2+\sigma_3^2$ over s gives $s=\bar{\sigma}$. □

Intuition

It is just "the average is the best single number to replace two numbers in the least-squares sense".

5-point solver. The essential matrix has 5 degrees of freedom, hence 5 correspondences suffice (Nistér 2004). This minimal solver is what runs inside RANSAC in COLMAP and OpenCV.

Beyond the slides

Minimal solvers are used inside RANSAC because the number of samples needed to hit an all-inlier set grows quickly with the sample size: with an inlier ratio $w = 0.5$, the probability that a random sample of 8 is all inliers is $w^8 \approx 0.4\%$ while for 5 it is $w^5 \approx 3.1\%$, so about $8\times$ fewer iterations are needed.

5.3.4 Decomposing the Essential Matrix *(slides 26)*

Write $\tilde{\boldsymbol{E}} = \boldsymbol{U}\,\mathrm{diag}(1,1,0)\boldsymbol{V}^\mathsf{T}$, where $\boldsymbol{u}_k, \boldsymbol{v}_k$ denote the k-th columns of $\boldsymbol{U}, \boldsymbol{V}$ with $\det\boldsymbol{U} = \det\boldsymbol{V} = +1$. Let

$$\boldsymbol{W} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Theorem 5.3.8: Four-fold ambiguity

$\tilde{\boldsymbol{E}} = [\boldsymbol{t}]_\times \boldsymbol{R}$ has exactly the four solutions (all up to scale)

$$\boldsymbol{R} \in \{\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T},\ \boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}\}, \qquad \boldsymbol{t} \in \{+\boldsymbol{u}_3,\ -\boldsymbol{u}_3\}, \qquad \|\boldsymbol{t}\| = 1.$$

Proof. For a unit vector $\boldsymbol{u}_3 = \boldsymbol{U}\boldsymbol{e}_3$ and $\det \boldsymbol{U} = 1$ we have $[\boldsymbol{u}_3]_\times = \boldsymbol{U}[\boldsymbol{e}_3]_\times \boldsymbol{U}^\mathsf{T}$, and $[\boldsymbol{e}_3]_\times = \operatorname{diag}(1,1,0)\boldsymbol{W}$ (direct multiplication). Hence $[\boldsymbol{u}_3]_\times = \boldsymbol{U}\operatorname{diag}(1,1,0)\boldsymbol{W}\boldsymbol{U}^\mathsf{T}$. Then

$$[\boldsymbol{u}_3]_\times (\boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}) = \boldsymbol{U}\operatorname{diag}(1,1,0)\boldsymbol{W}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T} = \tilde{\boldsymbol{E}}.$$

Moreover, $\boldsymbol{W}^2 = \operatorname{diag}(-1,-1,1)$ gives $[\boldsymbol{u}_3]_\times (\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}) = \boldsymbol{U}\operatorname{diag}(1,1,0)\boldsymbol{W}^2\boldsymbol{V}^\mathsf{T} = -\tilde{\boldsymbol{E}}$, i.e. $[-\boldsymbol{u}_3]_\times(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}) = \tilde{\boldsymbol{E}}$. So $(\boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}, +\boldsymbol{u}_3)$ and $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}, -\boldsymbol{u}_3)$ solve $\tilde{\boldsymbol{E}} = [\boldsymbol{t}]_\times \boldsymbol{R}$ exactly. Since $\tilde{\boldsymbol{E}}$ and $-\tilde{\boldsymbol{E}}$ are the same essential matrix (it is only defined up to scale, sign included), the pairs $(\boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}, -\boldsymbol{u}_3)$ and $(\boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T}, +\boldsymbol{u}_3)$ are equally valid. The null space of $[\boldsymbol{t}]_\times$ is $\operatorname{span}(\boldsymbol{t})$, which forces $\boldsymbol{t} \propto \boldsymbol{u}_3$, so there are no other solutions. Finally $\boldsymbol{R}' = \boldsymbol{U}\boldsymbol{W}\boldsymbol{V}^\mathsf{T} = \big(\boldsymbol{U}\operatorname{diag}(-1,-1,1)\boldsymbol{U}^\mathsf{T}\big)\boldsymbol{R}$ with $\boldsymbol{R} = \boldsymbol{U}\boldsymbol{W}^\mathsf{T}\boldsymbol{V}^\mathsf{T}$: it differs from $\boldsymbol{R}$ by a half turn about $\boldsymbol{u}_3 \propto \boldsymbol{t}$. □

Intuition

Two independent ambiguities hide in $\tilde{\boldsymbol{E}}$. (1) The *sign* of $\boldsymbol{t}$: $\tilde{\boldsymbol{E}}$ and $-\tilde{\boldsymbol{E}}$ are the same constraint, so we cannot tell if the second camera is left or right of the first from the epipolar constraint alone. (2) The *twisted pair*: rotating camera 2 by 180° about the baseline maps every epipolar plane to itself, so it also satisfies the constraint. $2 \times 2 = 4$ candidates; the geometry of the next slide tells us which one is real.

5.3.5 Geometry of the Four Solutions *(slides 27)*

With the true pose $(\boldsymbol{R}, \boldsymbol{t})$ and the other rotation $\boldsymbol{R}'$, the four candidates place a scene point as drawn in Fig. 19.

- **Baseline reversal** ($\boldsymbol{t} \to -\boldsymbol{t}$, left to right in the slide): moves the center of camera 2, $-\boldsymbol{R}^\mathsf{T}\boldsymbol{t}$, to the other side of camera 1. Every point lands behind both cameras.
- **Twisted pair** ($\boldsymbol{R} \to \boldsymbol{R}'$, top to bottom): $\boldsymbol{R}'^\mathsf{T}\boldsymbol{R} = \boldsymbol{U}\operatorname{diag}(-1,-1,1)\boldsymbol{U}^\mathsf{T}$, a half turn about $\boldsymbol{u}_3 \propto \boldsymbol{t}$: camera 2 turns by 180° about the baseline. Each point lands in front of one camera only.

Only $(\boldsymbol{R}, \boldsymbol{t})$ puts the scene points in front of both cameras.

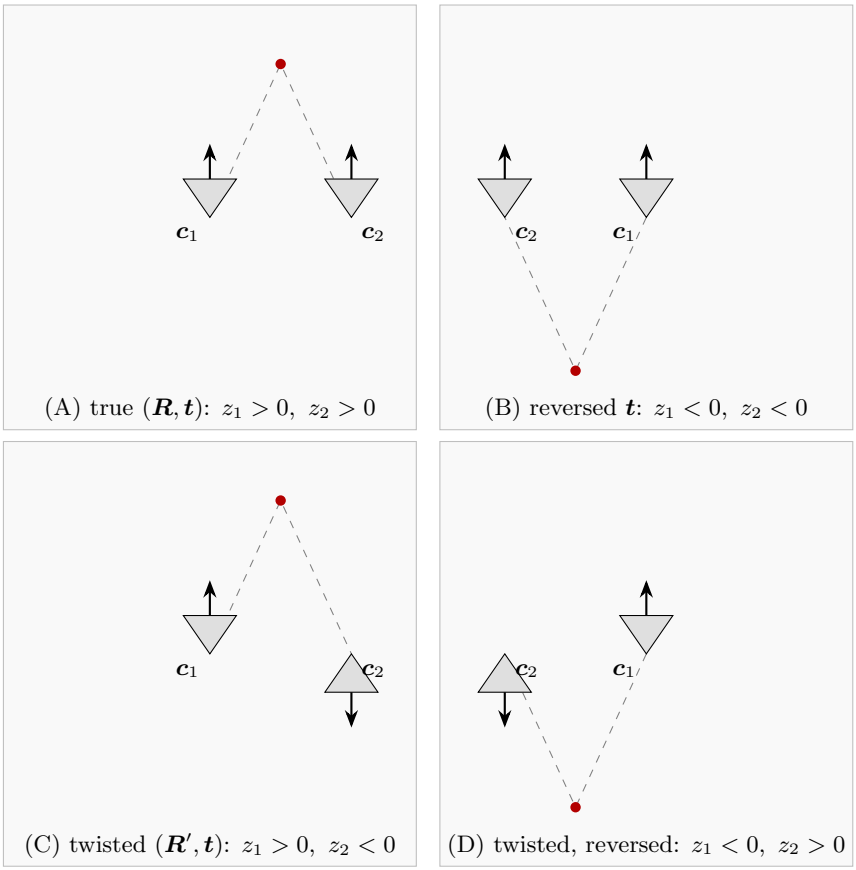


Figure 19: Schematic of the four candidate poses (after slide 27; in 2D, cameras drawn as triangles with an arrow along the viewing direction, red dot: reconstructed point, dashed: reconstructed rays). Only panel A has the point in front of both cameras.

Intuition

The point is the same in all four panels; what changes is where the two cameras "look". If I flip the baseline, I must also look in the opposite direction to see the same image, so the point appears behind me. If I twist camera 2 around the baseline, camera 2 now looks away from the scene: the point is behind it but still in front of camera 1.

5.3.6 Disambiguation by Cheirality *(slides 28)*

Cheirality: a reconstructed point must lie in front of both cameras. For each correspondence and each candidate $(\boldsymbol{R}_k, \boldsymbol{t}_k)$, the depths z_1, z_2 along the two rays follow from $\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$ with $\boldsymbol{x}_i = z_i \tilde{\boldsymbol{x}}_i$:

$$z_2 \tilde{\boldsymbol{x}}_2 = z_1 \boldsymbol{R}_k \tilde{\boldsymbol{x}}_1 + \boldsymbol{t}_k \iff \begin{bmatrix} -\boldsymbol{R}_k \tilde{\boldsymbol{x}}_1 & \tilde{\boldsymbol{x}}_2 \end{bmatrix} \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \boldsymbol{t}_k.$$

Three equations, two unknowns: solve by least squares.

Algorithm 5.3.9: Pose from the essential matrix

1. Decompose $\tilde{\boldsymbol{E}}$ into the four candidates (Thm. 5.3.8).
2. For each candidate and each correspondence solve the least-squares system above.
3. Keep the candidate with the most correspondences with $z_1 > 0$ and $z_2 > 0$. Count over *all* points: noise and points near infinity can flip individual signs.

Result: $[\boldsymbol{R}|\boldsymbol{t}]$, metric up to one global scale, fixed here by the convention $\|\boldsymbol{t}\| = 1$; later by the gauge of bundle adjustment (5.4) or by a normalization (5.5). In OpenCV: `findEssentialMat` (5-point solver inside RANSAC), then `recoverPose` (decomposition and this test).

Example 5.3.10: Cheirality in numbers

Use the rectified example: $u_1 = 0.1$, $u_2 = 0.3$, $\boldsymbol{R} = \boldsymbol{I}$. For $\boldsymbol{t}_k = (1,0,0)^\mathsf{T}$ the system $\begin{bmatrix}-\tilde{\boldsymbol{x}}_1 & \tilde{\boldsymbol{x}}_2\end{bmatrix}(z_1, z_2)^\mathsf{T} = \boldsymbol{t}_k$ with $\tilde{\boldsymbol{x}}_1 = (0.1, v, 1)^\mathsf{T}$, $\tilde{\boldsymbol{x}}_2 = (0.3, v, 1)^\mathsf{T}$ reads $-0.1z_1 + 0.3z_2 = 1$, $-vz_1 + vz_2 = 0$, $-z_1 + z_2 = 0$, so $z_1 = z_2 = z$ and $0.2z = 1$, $z = 5 > 0$: accepted. For $\boldsymbol{t}_k = (-1,0,0)^\mathsf{T}$ we get $0.2z = -1$, $z = -5 < 0$: the point would be behind both cameras: rejected.

5.3.7 Estimating the Epipolar Geometry with Unknown Intrinsics *(slides 29–30)*

If the calibration is unknown, we cannot use the local ray directions $\tilde{\boldsymbol{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$ and only know the pixel coordinates $\bar{\boldsymbol{x}}_i$.

Definition 5.3.11: Fundamental matrix

Substituting $\tilde{\boldsymbol{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$ in the epipolar constraint gives

$$\tilde{\boldsymbol{x}}_2^\mathsf{T}\tilde{\boldsymbol{E}}\tilde{\boldsymbol{x}}_1 = \bar{\boldsymbol{x}}_2^\mathsf{T}\underbrace{\boldsymbol{K}_2^{-\mathsf{T}}\tilde{\boldsymbol{E}}\boldsymbol{K}_1^{-1}}_{\tilde{\boldsymbol{F}}}\bar{\boldsymbol{x}}_1 = 0.$$

$\tilde{\boldsymbol{F}}$ is the *fundamental matrix*. Like $\tilde{\boldsymbol{E}}$ it is (without noise) of rank two and the epipoles can be recovered in the same way (as null vectors). The epipolar line of $\bar{\boldsymbol{x}}_1$ in image 2 is $\tilde{\boldsymbol{l}}_2 = \tilde{\boldsymbol{F}}\bar{\boldsymbol{x}}_1$.

The intrinsic parameters cannot be directly determined from $\tilde{\boldsymbol{F}}$ alone (a learned model that estimates the field of view directly from the images appears in 5.5).

Intuition

$\tilde{\boldsymbol{F}}$ is just $\tilde{\boldsymbol{E}}$ "dressed" in the unknown camera matrices: the same geometry, but expressed in pixels. It has 7 degrees of freedom (rank 2 and defined up to scale, $9 - 1 - 1$) instead of 5, because two extra parameters are not tied to a rigid motion. It is enough to draw epipolar lines and reject outliers, but you need $\boldsymbol{K}_i$ to convert it into a metric pose.

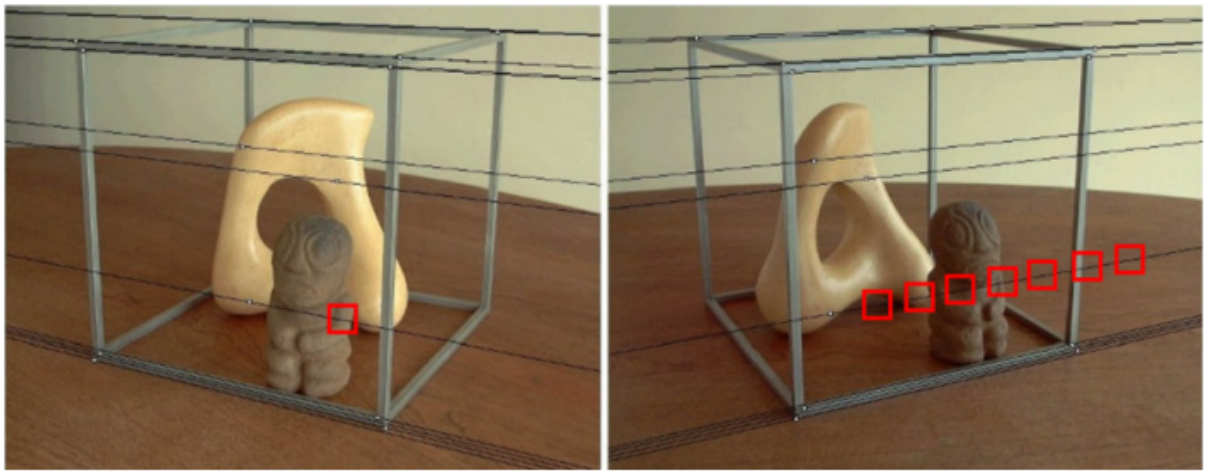


Figure 20: Epipolar lines for two real images based on the estimated epipolar geometry; corresponding points lie on the corresponding epipolar lines.

5.3.8 Triangulation *(slides 31–35)*

Problem (slide 32). Given the camera intrinsics and extrinsics, how can we recover 3D geometry? With noisy 2D observations $\bar{\boldsymbol{x}}_1, \bar{\boldsymbol{x}}_2$ the two rays might not intersect; we want the 3D point $\boldsymbol{x}$ that is, in some sense, closest to the two rays (Fig. 21).

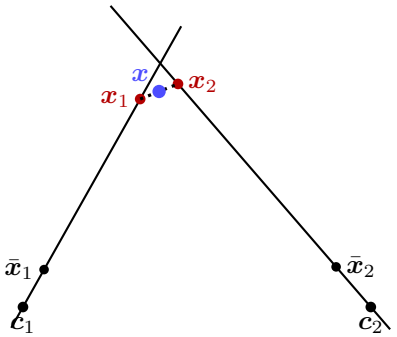


Figure 21: Two noisy rays do not intersect; $\boldsymbol{x}_1, \boldsymbol{x}_2$ are their closest points and $\boldsymbol{x}$ their midpoint (slide 32).

Proposition 5.3.12: Midpoint triangulation

Let the rays be $\boldsymbol{c}_i + s_i \boldsymbol{d}_i$ ($\|\boldsymbol{d}_i\| = 1$) and $\boldsymbol{b} = \boldsymbol{c}_2 - \boldsymbol{c}_1$. The closest points have parameters solving

$$\begin{pmatrix} 1 & -\boldsymbol{d}_1^\mathsf{T}\boldsymbol{d}_2 \\ \boldsymbol{d}_1^\mathsf{T}\boldsymbol{d}_2 & -1 \end{pmatrix} \begin{pmatrix} s_1 \\ s_2 \end{pmatrix} = \begin{pmatrix} \boldsymbol{d}_1^\mathsf{T}\boldsymbol{b} \\ \boldsymbol{d}_2^\mathsf{T}\boldsymbol{b} \end{pmatrix},$$

and the midpoint $\boldsymbol{x} = \frac{1}{2}(\boldsymbol{x}_1 + \boldsymbol{x}_2)$ minimizes the sum of squared distances to the two rays. The system degenerates exactly when $\boldsymbol{d}_1 \parallel \boldsymbol{d}_2$.

Proof. Minimize $\|\boldsymbol{c}_1 + s_1\boldsymbol{d}_1 - \boldsymbol{c}_2 - s_2\boldsymbol{d}_2\|^2$. Setting the derivatives in s_1, s_2 to zero gives $\boldsymbol{d}_1^\mathsf{T}(s_1\boldsymbol{d}_1 - s_2\boldsymbol{d}_2 - \boldsymbol{b}) = 0$ and $\boldsymbol{d}_2^\mathsf{T}(s_1\boldsymbol{d}_1 - s_2\boldsymbol{d}_2 - \boldsymbol{b}) = 0$, which is the stated system. Its determinant is $-1 + (\boldsymbol{d}_1^\mathsf{T}\boldsymbol{d}_2)^2 = -\sin^2\gamma$, where γ is the angle between the rays: zero iff the rays are parallel. □

Intuition

Two rays in 3D generically miss each other by a small gap; the segment joining their closest points is perpendicular to both. Taking its midpoint is the natural compromise. The determinant $\sin^2\gamma$ already hints at the next topic: when the rays are nearly parallel the problem becomes ill-conditioned.

Linear triangulation (DLT, slide 33). Let $\tilde{\boldsymbol{x}}_i^s = \tilde{\boldsymbol{P}}_i \tilde{\boldsymbol{x}}^w$ denote the projection of a 3D world point onto the image of the i-th camera. Both sides are homogeneous: they have the same direction but may differ in magnitude, so we consider the cross product $\bar{\boldsymbol{x}}_i^s \times \tilde{\boldsymbol{P}}_i \tilde{\boldsymbol{x}}^w = 0$. With $\tilde{\boldsymbol{p}}_{ik}^\mathsf{T}$ the k-th row of $\tilde{\boldsymbol{P}}_i$ this gives

$$\underbrace{\begin{pmatrix} x_i^s\,\tilde{\boldsymbol{p}}_{i3}^\mathsf{T} - \tilde{\boldsymbol{p}}_{i1}^\mathsf{T} \\ y_i^s\,\tilde{\boldsymbol{p}}_{i3}^\mathsf{T} - \tilde{\boldsymbol{p}}_{i2}^\mathsf{T} \end{pmatrix}}_{\boldsymbol{A}_i} \tilde{\boldsymbol{x}}^w = 0.$$

Stacking $N \geq 2$ observations yields $\boldsymbol{A}\tilde{\boldsymbol{x}}^w = 0$. As $\tilde{\boldsymbol{x}}^w$ is homogeneous this is a constrained least-squares problem; the solution is the right singular vector of $\boldsymbol{A}$ with the smallest singular value (the Direct Linear Transformation, as for homography estimation in lecture 01).

Intuition

Each view contributes two independent linear equations (the third row of the cross product is a combination of the other two), and the unknown 3D point has 3 degrees of freedom, so two

views already give $4 \geq 3$ equations. A single image of a point removes two degrees of freedom (it constrains the point to a line); the second one removes the rest and adds one redundant equation to absorb noise.

Reprojection error minimization (slide 34). While DLT often works well, it is not invariant to projective transformations. The gold standard is to minimize the reprojection error numerically,

$$\bar{\boldsymbol{x}}_w^* = \arg\min_{\bar{\boldsymbol{x}}_w} \sum_{i=1}^{N} \|\bar{\boldsymbol{x}}_i^s(\bar{\boldsymbol{x}}_w) - \bar{\boldsymbol{x}}_i^o\|_2^2,$$

with observations $\bar{\boldsymbol{x}}_i^o$. This allows taking measurement noise appropriately into account. For two views the minimum can also be obtained in closed form as the solution of a sixth degree polynomial (Hartley and Zisserman, Section 12.5).

Triangulation uncertainty (slide 35). Triangulation works differently well depending on the relative camera pose: uncertainty (shaded region) increases as the rays become more parallel. There is a **trade-off**: feature matching is easier for nearby views, but triangulation is harder. (A learned prior can also reconstruct views with little or no overlap, see 5.5.)

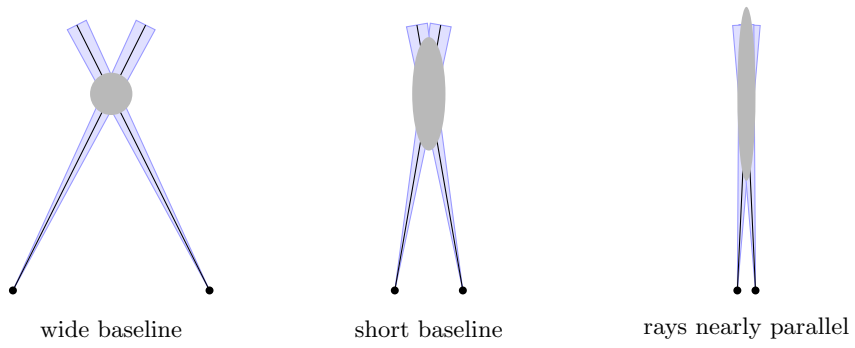


Figure 22: Schematic of triangulation uncertainty (after slide 35): the grey region is where the two noisy rays are consistent with the observations. It grows as the rays become parallel. The ellipses are drawn by hand, not computed.

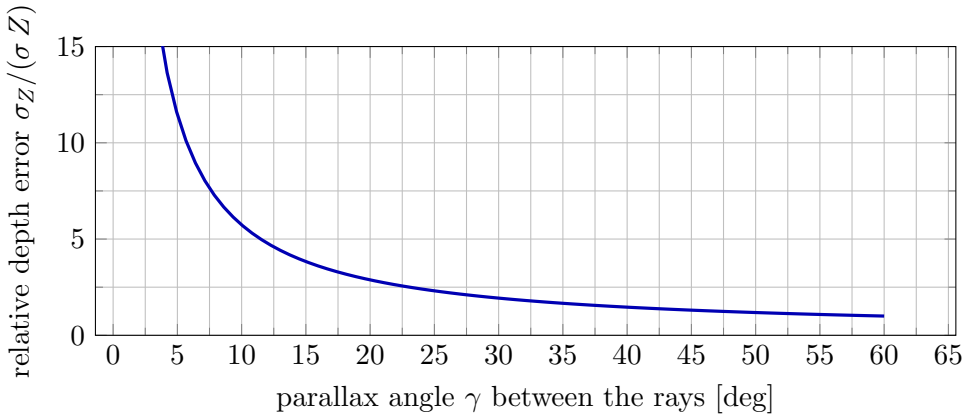


Figure 23: Original illustration of the trade-off. For a point at distance Z seen under angular noise σ per ray, the depth error is, to first order, $\sigma_Z \approx \sigma Z/(2\sin(\gamma/2)) \approx \sigma Z/\gamma$ (small γ). Below about 5° the error explodes; above about 30° it is flat, but matching becomes hard.

Intuition

Hold a finger at arm's length and look with one eye at a time: it jumps against the background. Do the same with a mountain: nothing moves. The jump is the parallax. Parallax is the signal that triangulation measures; if it is tiny, noise swamps it. Meanwhile large viewpoint changes make appearance very different, so matching fails. SfM pipelines therefore choose their image pairs carefully ('enough baseline for triangulation', slide 48).

5.4 Bundle Adjustment *(slides 36–54)*

5.4.1 Bundle Adjustment *(slides 37–39)*



Figure 24: Photo collection of Notre-Dame (left) and the resulting sparse reconstruction with camera frusta (right) (Snavely, Seitz and Szeliski, SIGGRAPH 2006).

Goal: optimize the reprojection errors (distance between an observed feature and the projected 3D point in the image plane) with respect to the camera parameters and the 3D point cloud.

Definition 5.4.1: Bundle adjustment

Let $\mathcal{T} = \{\boldsymbol{T}_i\}$ with $\boldsymbol{T}_i \in \mathrm{SE}(3)$ $(i = 1..N)$ denote the camera poses; the calibrations $\boldsymbol{K}_i$ are known and fixed. Let $\mathcal{X}_w = \{\boldsymbol{x}_p^w\}$, $\boldsymbol{x}_p^w \in \mathbb{R}^3$ $(p = 1..P)$ be the 3D points in world coordinates and $\mathcal{X}_s = \{\boldsymbol{x}_{ip}^s\}$, $\boldsymbol{x}_{ip}^s \in \mathbb{R}^2$ the image observations in all cameras. Bundle adjustment minimizes the reprojection error of all observations:

$$\mathcal{T}^*, \mathcal{X}_w^* = \arg\min_{\mathcal{T},\mathcal{X}_w} \sum_{i=1}^{N} \sum_{p=1}^{P} w_{ip} \left\| \boldsymbol{x}_{ip}^s - \pi_i(\boldsymbol{x}_p^w) \right\|_2^2,$$

where w_{ip} indicates whether point p is observed in image i and $\pi_i(\boldsymbol{x}_p^w)$ is the projection of the 3D world point into camera i with its pose $\boldsymbol{T}_i = [\boldsymbol{R}_i|\boldsymbol{t}_i]$ and calibration $\boldsymbol{K}_i$:

$$\pi_i(\boldsymbol{x}_p^w) = \begin{pmatrix} \tilde{x}_{ip}^s / \tilde{w}_{ip}^s \\ \tilde{y}_{ip}^s / \tilde{w}_{ip}^s \end{pmatrix}, \qquad \tilde{\boldsymbol{x}}_{ip}^s = \boldsymbol{K}_i(\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i).$$

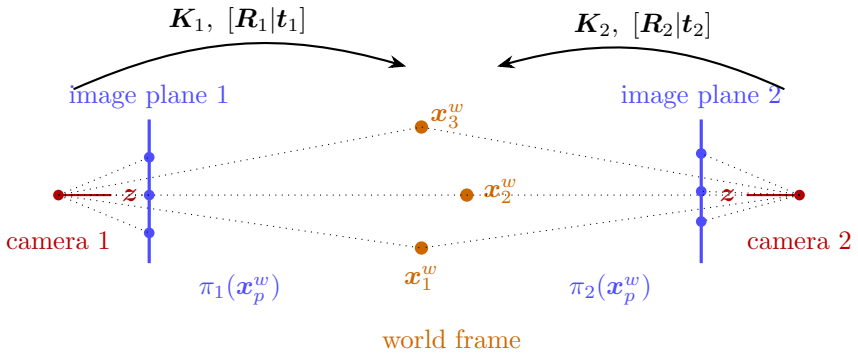


Figure 25: The bundle adjustment setup (slide 39): the 3D points are projected into both cameras by π_i. With the $\boldsymbol{K}_i$ fixed, bundle adjustment optimizes the poses $\mathcal{T}$ and the points $\mathcal{X}_w$ jointly.

Intuition

Imagine each 3D point and each camera attached to the others by elastic bands: every observation is a band pulling the *predicted* pixel towards the *measured* pixel. The name "bundle" refers to the bundles of light rays between each camera center and its points. Adjusting them all at once (poses and points) finds the configuration where the total tension (squared reprojection error) is lowest. This is the maximum-likelihood estimate under independent isotropic Gaussian pixel noise.

5.4.2 Challenges of Bundle Adjustment *(slides 40)*

- **Initialization.** The energy landscape is highly non-convex: a good initialization is crucial to avoid bad local minima. As initializing all 3D points and cameras jointly is difficult (occlusion, viewpoint, matching outliers), *incremental* bundle adjustment starts from a carefully selected two-view reconstruction and iteratively adds new images/cameras.
- **Optimization.** With millions of features and thousands of cameras, large-scale bundle adjustment is computationally demanding (cubic complexity in the number of unknowns). Luckily the problem is *sparse* (not every 3D point is observed in every camera), and efficient sparse implementations (e.g. g2o) can be exploited.
- Here the calibrations $\boldsymbol{K}_i$ are fixed; bundle adjustment can also estimate one $\boldsymbol{K}_i$ per camera.
- Predicting cameras and points with a network removes the incremental initialization (5.5).

5.4.3 Bundle Adjustment as Nonlinear Least Squares *(slides 41)*

Collect the unknowns in $\mathcal{S} = (\boldsymbol{T}_1, \dots, \boldsymbol{T}_N, \boldsymbol{x}_1^w, \dots, \boldsymbol{x}_P^w)$ with $\boldsymbol{T}_i = \begin{pmatrix} \boldsymbol{R}_i & \boldsymbol{t}_i \\ \boldsymbol{0}^\intercal & 1 \end{pmatrix} \in \mathrm{SE}(3)$ and $\boldsymbol{x}_p^w \in \mathbb{R}^3$. Each residual compares the observation with the projection of the point, moved into camera i by its pose:

$$E(\mathcal{S}) = \sum_{i,p} w_{ip} \|\boldsymbol{r}_{ip}\|_2^2, \qquad \boldsymbol{r}_{ip} = \boldsymbol{x}_{ip}^s - \pi\big(\boldsymbol{K}_i(\boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i)\big),$$

with the perspective division $\pi(\tilde{x}, \tilde{y}, \tilde{w}) = (\tilde{x}/\tilde{w}, \tilde{y}/\tilde{w})$. Stacking all residuals into $\boldsymbol{r}(\mathcal{S})$ gives $E(\mathcal{S}) = \|\boldsymbol{r}(\mathcal{S})\|_2^2$.

- π_i contains a rotation and a perspective division: there is no closed form, and the minimization iterates from an initialization.
- The poses do not form a vector space, so each iteration must update them on SE(3).
- E does not change under a global rotation, translation and scale of all poses and points: these 7 degrees of freedom are fixed by holding one pose and one distance.

Proposition 5.4.2: Gauge freedom

Let $(\boldsymbol{R}_i, \boldsymbol{t}_i, \boldsymbol{x}_p^w)$ be any solution. For any rotation $\boldsymbol{Q}$, translation $\boldsymbol{a}$ and scale $\alpha > 0$, define $\boldsymbol{x}_p'^w = \alpha \boldsymbol{Q}\boldsymbol{x}_p^w + \boldsymbol{a}$, $\boldsymbol{R}_i' = \boldsymbol{R}_i\boldsymbol{Q}^\mathsf{T}$, $\boldsymbol{t}_i' = \alpha\boldsymbol{t}_i - \boldsymbol{R}_i\boldsymbol{Q}^\mathsf{T}\boldsymbol{a}$. Then every projection, and hence E, is unchanged. Consequently E is constant along a 7-dimensional family of solutions (the Hessian has a 7-dimensional null space).

Proof. $\boldsymbol{R}_i'\boldsymbol{x}_p'^w + \boldsymbol{t}_i' = \alpha\boldsymbol{R}_i\boldsymbol{x}_p^w + \boldsymbol{R}_i\boldsymbol{Q}^\mathsf{T}\boldsymbol{a} + \alpha\boldsymbol{t}_i - \boldsymbol{R}_i\boldsymbol{Q}^\mathsf{T}\boldsymbol{a} = \alpha(\boldsymbol{R}_i\boldsymbol{x}_p^w + \boldsymbol{t}_i)$, so each camera-frame point is only multiplied by α. Since a pixel is a ratio of coordinates, π_i is invariant to scaling its argument, and all residuals are unchanged. The family has $3+3+1=7$ parameters. □

Intuition

The images do not know where the world origin is, which way is "north", or whether the scene is a toy or a real building (the scale ambiguity of Section 5.3). Fixing *one pose* (6 dof: $\boldsymbol{T}_1 = \boldsymbol{I}$) plus *one distance* (1 dof: e.g. $\|\boldsymbol{t}_2\| = 1$) removes the 7 free directions; this is precisely the "gauge" chosen in the initialization of incremental SfM (slide 48).

5.4.4 Updates in the Tangent Space *(slides 42)*

A pose cannot be updated by addition: $\boldsymbol{T}_i + \Delta\boldsymbol{T}$ is in general not a rigid transformation, since $\boldsymbol{R}^\mathsf{T}\boldsymbol{R} = \boldsymbol{I}$ breaks, and its 12 entries carry only 6 degrees of freedom. The update is instead a 6-vector in the Lie algebra $\mathfrak{se}(3) \cong \mathbb{R}^6$, the tangent space of SE(3), mapped onto the group by the exponential map. Points live in $\mathbb{R}^3$ and are added:

$$\boldsymbol{T}_i \leftarrow \mathrm{Exp}(\Delta\boldsymbol{\xi}_i)\,\boldsymbol{T}_i, \qquad \boldsymbol{x}_p^w \leftarrow \boldsymbol{x}_p^w + \Delta\boldsymbol{x}_p, \qquad \mathrm{Exp}(\Delta\boldsymbol{\xi}) = \exp\begin{pmatrix}[\Delta\boldsymbol{\omega}]_\times & \Delta\boldsymbol{\rho} \\ \boldsymbol{0}^\mathsf{T} & 0\end{pmatrix}, \quad \Delta\boldsymbol{\xi} = (\Delta\boldsymbol{\rho}, \Delta\boldsymbol{\omega}),$$

where $\Delta\boldsymbol{\omega}$ is an axis-angle rotation, $\Delta\boldsymbol{\rho}$ the translational part and exp the matrix exponential. The update is *minimal and unconstrained*: 6 numbers per pose, and every update stays in SE(3).

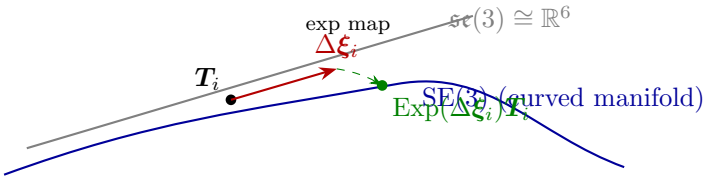


Figure 26: Local update on a curved space: take a step $\Delta\boldsymbol{\xi}_i$ in the flat tangent space, then map it back onto the group with the exponential map (schematic).

Beyond the slides

For $\theta = \|\Delta\boldsymbol{\omega}\|$ the exponential has a closed form (Rodrigues): $\exp[\Delta\boldsymbol{\omega}]_\times = \boldsymbol{I} + \frac{\sin\theta}{\theta}[\Delta\boldsymbol{\omega}]_\times + \frac{1-\cos\theta}{\theta^2}[\Delta\boldsymbol{\omega}]_\times^2$, and $\mathrm{Exp}(\Delta\boldsymbol{\xi}) = \begin{pmatrix}\exp[\Delta\boldsymbol{\omega}]_\times & \boldsymbol{V}\Delta\boldsymbol{\rho} \\ \boldsymbol{0}^\mathsf{T} & 1\end{pmatrix}$ with $\boldsymbol{V} = \boldsymbol{I} + \frac{1-\cos\theta}{\theta^2}[\Delta\boldsymbol{\omega}]_\times + \frac{\theta-\sin\theta}{\theta^3}[\Delta\boldsymbol{\omega}]_\times^2$.

Intuition

Think of walking on the surface of the Earth: you plan a step on a flat map (the tangent plane) and then "wrap" it back onto the sphere. If you added numbers straight into the 12 matrix entries you would step *off* the surface (the matrix stops being a rotation).

5.4.5 The Gauss–Newton Algorithm *(slides 43)*

Each residual depends on one pose and one point. Around the current estimate it is linearized in their increments:

$$\boldsymbol{r}_{ip}\big(\mathrm{Exp}(\Delta\boldsymbol{\xi}_i)\boldsymbol{T}_i,\ \boldsymbol{x}_p^w + \Delta\boldsymbol{x}_p\big) \approx \boldsymbol{r}_{ip} + \boldsymbol{J}_{ip}^{\xi}\Delta\boldsymbol{\xi}_i + \boldsymbol{J}_{ip}^{x}\Delta\boldsymbol{x}_p.$$

Lemma 5.4.3: Jacobians of the reprojection residual

With $\boldsymbol{x}_{ip}^c = \boldsymbol{R}_i \boldsymbol{x}_p^w + \boldsymbol{t}_i$ and $\boldsymbol{J}_\pi \in \mathbb{R}^{2\times 3}$ the Jacobian of the projection $\boldsymbol{x}^c \mapsto \pi(\boldsymbol{K}_i \boldsymbol{x}^c)$ (so $\boldsymbol{J}_\pi = (\partial\pi/\partial\tilde{\boldsymbol{x}})\boldsymbol{K}_i$), both blocks at zero increment are

$$\boldsymbol{J}_{ip}^{\xi} = -\boldsymbol{J}_\pi \begin{bmatrix} \boldsymbol{I} & -[\boldsymbol{x}_{ip}^c]_\times \end{bmatrix} \in \mathbb{R}^{2\times 6}, \qquad \boldsymbol{J}_{ip}^{x} = -\boldsymbol{J}_\pi \boldsymbol{R}_i \in \mathbb{R}^{2\times 3}.$$

Proof. For a small $\Delta\boldsymbol{\xi} = (\Delta\boldsymbol{\rho}, \Delta\boldsymbol{\omega})$, $\mathrm{Exp}(\Delta\boldsymbol{\xi}) \approx \boldsymbol{I} + \begin{pmatrix} [\Delta\boldsymbol{\omega}]_\times & \Delta\boldsymbol{\rho} \\ \boldsymbol{0}^\mathsf{T} & 0 \end{pmatrix}$, so the camera-frame point becomes $\boldsymbol{x}^c + \Delta\boldsymbol{\omega} \times \boldsymbol{x}^c + \Delta\boldsymbol{\rho} = \boldsymbol{x}^c + \Delta\boldsymbol{\rho} - [\boldsymbol{x}^c]_\times \Delta\boldsymbol{\omega}$. The residual is $\boldsymbol{x}^s - \pi(\boldsymbol{K}\boldsymbol{x}^c)$, so by the chain rule $\partial\boldsymbol{r}/\partial\Delta\boldsymbol{\xi} = -\boldsymbol{J}_\pi[\boldsymbol{I} \quad -[\boldsymbol{x}^c]_\times]$. A change $\Delta\boldsymbol{x}$ of the world point changes $\boldsymbol{x}^c$ by $\boldsymbol{R}_i \Delta\boldsymbol{x}$, giving $-\boldsymbol{J}_\pi \boldsymbol{R}_i$. □

Intuition

The first block says: moving the camera by $\Delta\boldsymbol{\rho}$ is the same as moving the point by $\Delta\boldsymbol{\rho}$ in the camera frame, while rotating by $\Delta\boldsymbol{\omega}$ moves the point sideways by an amount proportional to its distance ($\Delta\boldsymbol{\omega} \times \boldsymbol{x}^c$): far points are sensitive to rotation, near points to translation. The leading minus sign comes from $\boldsymbol{r}$ = observed − predicted.

With all increments stacked into $\Delta\boldsymbol{s} = (\Delta\boldsymbol{\xi}_1, \ldots, \Delta\boldsymbol{\xi}_N, \Delta\boldsymbol{x}_1, \ldots, \Delta\boldsymbol{x}_P) \in \mathbb{R}^{6N+3P}$, minimizing $\|\boldsymbol{r} + \boldsymbol{J}\Delta\boldsymbol{s}\|_2^2$ gives the normal equations; the update is repeated until convergence:

$$\boldsymbol{H}\Delta\boldsymbol{s} = -\boldsymbol{b}, \quad \boldsymbol{H} = \boldsymbol{J}^\mathsf{T}\boldsymbol{J}, \quad \boldsymbol{b} = \boldsymbol{J}^\mathsf{T}\boldsymbol{r}, \qquad \boldsymbol{T}_i \leftarrow \mathrm{Exp}(\Delta\boldsymbol{\xi}_i)\boldsymbol{T}_i, \quad \boldsymbol{x}_p^w \leftarrow \boldsymbol{x}_p^w + \Delta\boldsymbol{x}_p.$$

Algorithm 5.4.4: Gauss–Newton for bundle adjustment

1. Start from an initialization $\mathcal{S}_0$ (e.g. two-view reconstruction).
2. Compute all residuals $\boldsymbol{r}_{ip}$ and Jacobians (Lemma 5.4.3).
3. Build $\boldsymbol{H} = \boldsymbol{J}^\mathsf{T}\boldsymbol{J}$ and $\boldsymbol{b} = \boldsymbol{J}^\mathsf{T}\boldsymbol{r}$ and solve $\boldsymbol{H}\Delta\boldsymbol{s} = -\boldsymbol{b}$ (exploiting sparsity, next subsection).
4. Update poses on SE(3) and points in $\mathbb{R}^3$; repeat steps 2–4 until $\|\Delta\boldsymbol{s}\|$ or the decrease of E is small.

Beyond the slides

In practice, Levenberg–Marquardt is used: $(\boldsymbol{H} + \lambda\boldsymbol{D})\Delta\boldsymbol{s} = -\boldsymbol{b}$ with a damping λ that interpolates between Gauss–Newton ($\lambda \to 0$, fast near the minimum) and gradient descent (λ large, safe far away). Robust loss functions (Huber, Cauchy) are commonly applied to reduce the effect of outliers. COLMAP uses the solver Ceres (slide 51).

5.4.6 The Schur Complement *(slides 44)*

With the pose increments $\Delta\boldsymbol{\xi} \in \mathbb{R}^{6N}$ ordered before the point increments $\Delta\boldsymbol{x} \in \mathbb{R}^{3P}$, the normal equations read

$$\begin{pmatrix} \boldsymbol{H}_{\xi\xi} & \boldsymbol{H}_{\xi x} \\ \boldsymbol{H}_{x\xi} & \boldsymbol{H}_{xx} \end{pmatrix} \begin{pmatrix} \Delta\boldsymbol{\xi} \\ \Delta\boldsymbol{x} \end{pmatrix} = - \begin{pmatrix} \boldsymbol{b}_\xi \\ \boldsymbol{b}_x \end{pmatrix}.$$

$\boldsymbol{H}_{xx}$ is block diagonal, one 3×3 block per point, so its inverse is cheap. Eliminating the points leaves a system in the $6N$ pose increments alone, and back substitution recovers the points.

Theorem 5.4.5: Schur complement reduction

If $\boldsymbol{H}_{xx}$ is invertible, the pose increments solve

$$(\boldsymbol{H}_{\xi\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{H}_{x\xi})\Delta\boldsymbol{\xi} = -(\boldsymbol{b}_\xi - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{b}_x), \qquad \Delta\boldsymbol{x} = -\boldsymbol{H}_{xx}^{-1}(\boldsymbol{b}_x + \boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi}),$$

and these give the same pose update as the full system.

Proof. The second block row reads $\boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi} + \boldsymbol{H}_{xx}\Delta\boldsymbol{x} = -\boldsymbol{b}_x$, which gives $\Delta\boldsymbol{x}$ as stated. Substituting it into the first block row, $\boldsymbol{H}_{\xi\xi}\Delta\boldsymbol{\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}(\boldsymbol{b}_x + \boldsymbol{H}_{x\xi}\Delta\boldsymbol{\xi}) = -\boldsymbol{b}_\xi$, and rearranging gives the reduced system. □

The matrix on the left, $\boldsymbol{S} = \boldsymbol{H}_{\xi\xi} - \boldsymbol{H}_{\xi x}\boldsymbol{H}_{xx}^{-1}\boldsymbol{H}_{x\xi}$, is the Schur complement of $\boldsymbol{H}_{xx}$ in $\boldsymbol{H}$. It yields the same pose update from $6N$ unknowns instead of $6N+3P$: thousands instead of millions. Each point is then updated independently.

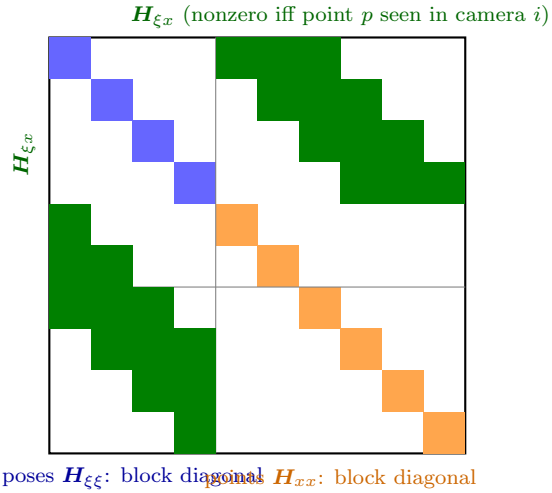


Figure 27: Arrowhead sparsity of the BA Hessian for $N = 4$ cameras and $P = 6$ points (original illustration). Cameras and points interact only if they share an observation. Eliminating the points (orange) leaves a small 4×4-block reduced camera system.

Example 5.4.6: Why the Schur complement matters

Take $N = 100$ cameras and $P = 100{,}000$ points (a modest dataset). The full system has $6N + 3P = 300{,}600$ unknowns, a dense solve would cost $\sim (3 \cdot 10^5)^3 \approx 2.7 \cdot 10^{16}$ operations. The reduced system has $6N = 600$ unknowns, costing $\sim 600^3 \approx 2.2 \cdot 10^8$ operations; the 10^5 small 3×3 inversions are trivial. That is a factor of about 10^8.

Intuition

The points never talk directly to each other, only through the cameras that see them. So we can "integrate out" each point (a tiny 3×3 problem), which replaces the point-to-camera coupling by an additional camera-to-camera coupling for every pair of cameras that share a point. After solving for the cameras, each point just reads the new camera positions and updates itself. (This is the standard trick behind *reduced camera systems* in Triggs et al. 2000.)

5.4.7 Incremental Structure-from-Motion *(slides 45–51)*

The feature matching and reconstruction pipeline (COLMAP, Schönberger and Frahm 2016) has two big stages (Fig. 28): **correspondence search** (find and match robust 2D features across images) and **incremental reconstruction** (start with two views and incrementally add cameras).

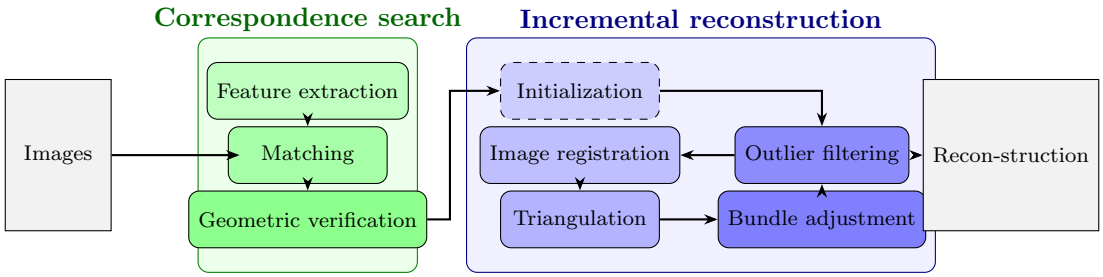


Figure 28: Incremental SfM pipeline of COLMAP (redrawn from slide 45). The registration–triangulation–filtering–bundle-adjustment loop runs once per added image.

Feature extraction (slide 46). Detect features (e.g. SIFT, SURF, BRISK) in all input images.

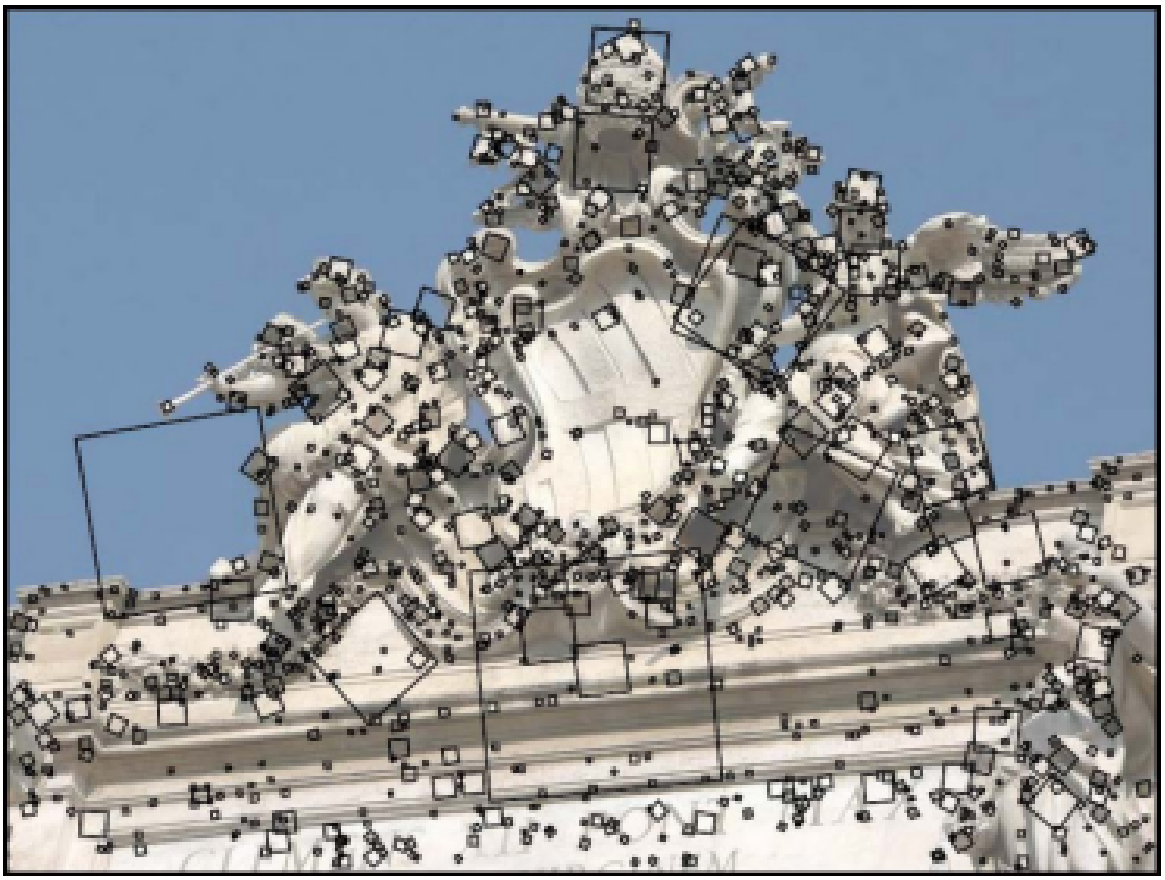


Figure 29: Features detected on a photograph of a monument: boxes show location, scale and orientation of each feature (Schönberger and Frahm, CVPR 2016).

Feature matching and geometric verification (slide 47). Find overlapping image pairs and the associated feature correspondences. Candidate matches are verified geometrically, i.e. they are kept only if they are consistent with an epipolar geometry or homography (RANSAC).

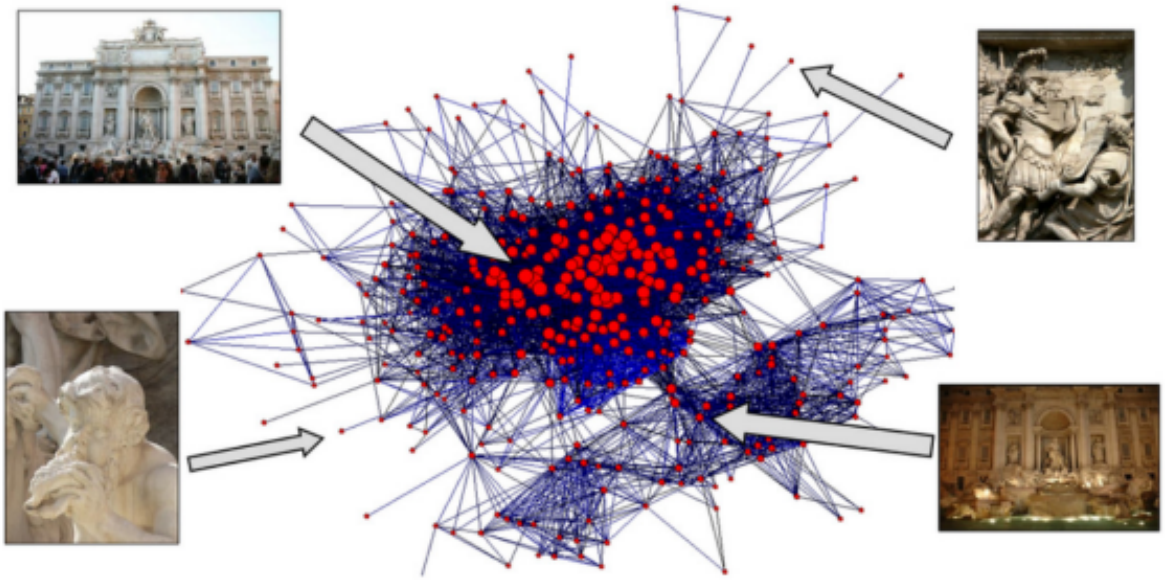


Figure 30: Graph of images (nodes) connected when they overlap; example photos of the Trevi fountain at different scales and lighting (Schönberger and Frahm, CVPR 2016).

Initialization (slide 48).

Algorithm 5.4.7: SfM initialization from two views

1. Select **two views** with many correspondences $\bar{\boldsymbol{x}}_1 \leftrightarrow \bar{\boldsymbol{x}}_2$ and enough baseline for triangulation.
2. Estimate $\tilde{\boldsymbol{E}}$ from $\tilde{\boldsymbol{x}}_2^\mathsf{T} \tilde{\boldsymbol{E}} \tilde{\boldsymbol{x}}_1 = 0$, with $\tilde{\boldsymbol{x}}_i = \boldsymbol{K}_i^{-1} \bar{\boldsymbol{x}}_i$, by the 5-point solver inside RANSAC.
3. Decompose $\tilde{\boldsymbol{E}}$ into $[\boldsymbol{R}|\boldsymbol{t}]$ and keep the candidate that puts the points in front of both cameras (Alg. 5.3.9).
4. Fix the gauge: $\boldsymbol{T}_1 = \boldsymbol{I}$ and $\boldsymbol{T}_2 = [\boldsymbol{R}|\boldsymbol{t}]$ with $\|\boldsymbol{t}\| = 1$ (this removes the 7 gauge degrees of freedom, Prop. 5.4.2).
5. **Triangulate** the inliers to obtain the first points $\boldsymbol{x}_p^w$.

Image registration (slide 49). Find a new image with correspondences to the current set and estimate its camera pose. Let $\mathcal{X} = \{\bar{\boldsymbol{x}}_i^s, \bar{\boldsymbol{x}}_i^w\}_{i=1}^N$ be a set of 3D-to-2D correspondences related by $\bar{\boldsymbol{x}}_i^s = \boldsymbol{P}\bar{\boldsymbol{x}}_i^w$. As the vectors are homogeneous, they have the same direction but differ in magnitude, so the equation can be expressed as $\bar{\boldsymbol{x}}_i^s \times \boldsymbol{P}\bar{\boldsymbol{x}}_i^w = 0$. Using the Direct Linear Transform this is a linear equation in the entries of $\boldsymbol{P}$; the solution of the constrained system (fixing $\boldsymbol{P}$'s scale) is given by SVD. Given that $\boldsymbol{P} = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}]$ and $\boldsymbol{K}$ is upper-triangular, both $\boldsymbol{K}$ and $\boldsymbol{R}$ can be obtained from the 3×3 front submatrix of $\boldsymbol{P}$ by standard RQ factorization. If $\boldsymbol{K}$ is known, we can even estimate $\boldsymbol{P}$ from only three points (P3P). In practice, RANSAC is used to remove outliers.

Intuition

Registration is the "PnP" problem: given a few points whose 3D position is already known from earlier triangulation, find where the new camera stands. $\boldsymbol{P}$ has 11 degrees of freedom (a 3×4 matrix up to scale) and every 3D–2D pair provides two independent equations, so DLT needs at least 6 points ($11/2 = 5.5$, rounded up). If $\boldsymbol{K}$ is known only 6 unknowns remain, and 3 points (6 equations) suffice, at the price of up to four solutions (P3P). Triangulation (known cameras $\rightarrow$ points) and registration (known points $\rightarrow$ camera) alternate: this is how the reconstruction "crawls" outwards.

Triangulation (slide 50). Given the newly registered image, new correspondences can be triangulated. In COLMAP, a robust triangulation method handles outliers.

Bundle adjustment and outlier filtering (slide 51). Minimize the reprojection error of all observations with respect to all poses and 3D points (Def. 5.4.1). Since incremental SfM only affects the model locally, COLMAP performs *local* BA on the locally connected images, and *global* BA only once in a while for efficiency. For the sparse large-scale optimization it uses Ceres. After each BA, observations with large reprojection errors and cameras with abnormal fields of view or large distortion coefficients are removed.

Algorithm 5.4.8: Incremental SfM (COLMAP-style)

1. Extract features in all images, match overlapping pairs, verify geometrically.
2. Initialize with a two-view reconstruction (Alg. 5.4.7).
3. **Repeat** until no image can be added: register the best new image (DLT/P3P inside RANSAC); triangulate new points; run local BA; filter outliers (large reprojection error, abnormal cameras); occasionally run global BA.

Intuition

Because the cost is highly non-convex, you "grow a crystal": start from a tiny well-conditioned seed (two views) and add only what is consistent with it, re-optimizing after each addition so errors never accumulate far from where the optimizer can fix them. The price: images are registered one at a time, so an early mistake propagates (a limit discussed in 5.5).

5.4.8 Building Rome in a Day, COLMAP SfM and COLMAP MVS *(slides 52–54)*

- **Building Rome in a Day** (Agarwal et al., ICCV 2009): first large-scale SfM from unstructured images (slide 52).
- **COLMAP** significantly improves accuracy and robustness compared to prior work (slide 53).
- COLMAP features a second **multi-view stereo** stage to obtain dense geometry (slide 54; Schönberger et al., ECCV 2016). A feed-forward network predicts dense depth without known cameras (5.5), whereas MVS needs SfM first.



Figure 31: Sparse point cloud and camera frusta of the Colosseum reconstructed from internet photos (Agarwal, Snavely, Simon, Seitz and Szeliski, "Building Rome in a Day", ICCV 2009).

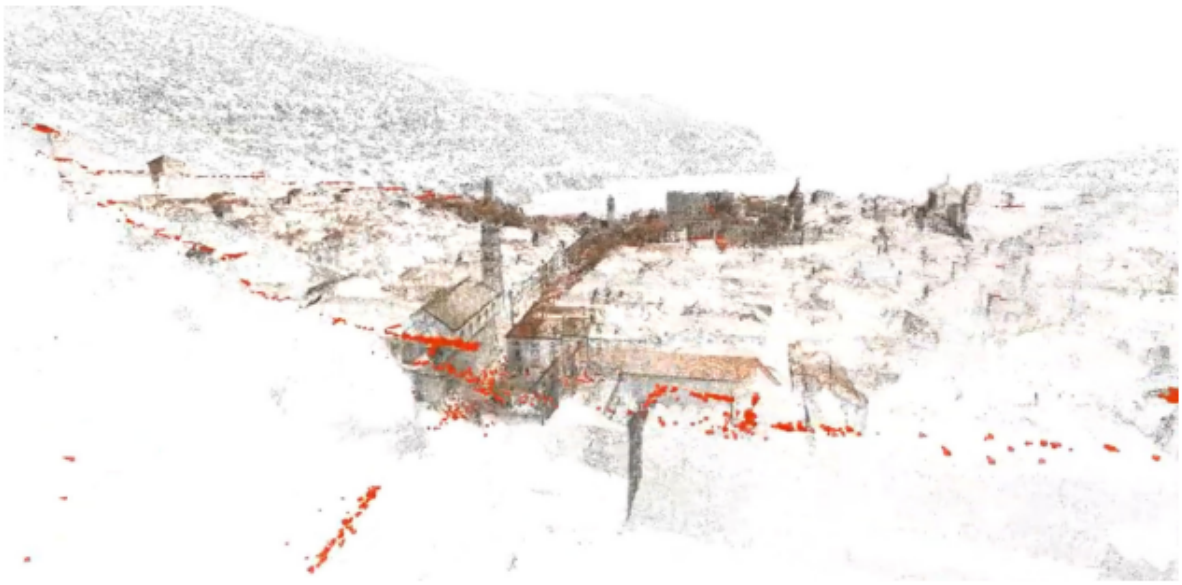


Figure 32: COLMAP sparse reconstruction of a city scene; red marks the camera positions (Schönberger and Frahm, CVPR 2016).



Figure 33: Dense COLMAP multi-view stereo reconstruction of a building (Schönberger, Zheng, Frahm and Pollefeys, ECCV 2016).

5.5 Feed-Forward Structure-from-Motion *(slides 55–68)*

5.5.1 Limits of Incremental Structure-from-Motion *(slides 56)*

Looking back at Fig. 28:

- every box is a separately engineered stage, with its own thresholds and failure cases;
- images are registered one at a time: a poor initial pair, outliers or drift propagate to every later image;

- runtime grows to hours for large photo collections; few views, low overlap or little texture leave too few correspondences;
- **Global SfM** estimates all cameras jointly without learning: rotation averaging, then camera positions, then one bundle adjustment. GLOMAP (Pan, Baráth, Pollefeys and Schönberger, ECCV 2024), a recent global pipeline, is on par with COLMAP in accuracy and orders of magnitude faster.

Intuition

Incremental SfM is like building a house brick by brick with one mason: reliable but slow, and a crooked first brick tilts the whole house. Global SfM hires many masons who work simultaneously and then reconcile. Feed-forward SfM, the topic of this section, replaces the masons by a single network that has seen millions of houses and sketches the whole building at once.

5.5.2 From Learned Components to Learned Reconstruction *(slides 57)*

Year	Method	What is learned
2018	SuperPoint	keypoints and their descriptors, from one network (5.2)
2020	SuperGlue	the matching of two keypoint sets, with a graph neural network
2021	LoFTR	dense matches between two images, with a transformer and no detector
2024	VGGSfM	a complete SfM pipeline, differentiable, trained end to end
2024	DUSt3R, MASt3R	pointmaps from two images (lecture 04); more views need a global alignment
2025	VGGT	cameras, depth, point maps and tracks for many views, in one forward pass

Table 2: Progressive replacement of the classical pipeline by learned components (slide 57).

Definition 5.5.1: Pointmap

For an $H \times W$ image $\boldsymbol{I}_a$, the pointmap $\hat{\boldsymbol{X}}^{a,1} \in \mathbb{R}^{H \times W \times 3}$ stores *one 3D point per pixel*, for both images $a \in \{1, 2\}$, expressed in the frame of camera 1.

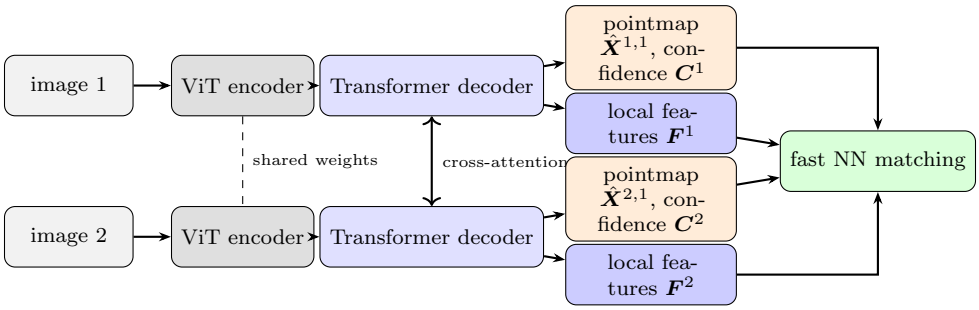


Figure 34: MASt3R-style two-view network (simplified from slide 57): tokens $\boldsymbol{H}^a$ from a ViT encoder are decoded with cross-attention into pointmaps with confidence (grey path) and local features (blue path), used for geometrical or feature-based matching.

Intuition

A pointmap is a depth map that already lives in a common world frame: once pixel (u, v) of image 2 and pixel (u', v') of image 1 carry (almost) the same 3D point, they are a correspondence *by construction*. DUSt3R and MASt3R use this to skip the detect-describe-match-verify chain, but for more than two images they still need a post-hoc global alignment; VGGT removes that last step.

5.5.3 Visual Geometry Grounded Transformer (VGGT) *(slides 58)*

- **Input:** one to hundreds of images of a scene; the first image defines the world frame.
- **Output,** in one forward pass of under a second: camera parameters, depth maps, point maps and point tracks for every image.
- Without post-processing, it often outperforms methods that refine their output by geometric optimization.



Figure 35: VGGT takes a set of images of a scene (left) and predicts, in one forward pass, camera poses, a dense point cloud, depth maps and point tracks (Wang et al., CVPR 2025).

5.5.4 Problem Definition *(slides 59)*

Definition 5.5.2: VGGT as a function

VGGT is one network with learned weights θ. It takes N images $\boldsymbol{I}_1, \ldots, \boldsymbol{I}_N$, each of $H \times W$ pixels, and predicts four outputs per image i (the hat marks a prediction):

$$f_\theta(\boldsymbol{I}_1, \ldots, \boldsymbol{I}_N) = \big(\hat{\boldsymbol{g}}_i,\ \hat{\boldsymbol{D}}_i,\ \hat{\boldsymbol{X}}_i,\ \boldsymbol{F}_i\big)_{i=1}^{N}.$$

- **Camera** $\hat{\boldsymbol{g}}_i \in \mathbb{R}^9$: rotation quaternion $\hat{\boldsymbol{q}}_i \in \mathbb{R}^4$, translation $\hat{\boldsymbol{t}}_i \in \mathbb{R}^3$ and field of view $\hat{\boldsymbol{\phi}}_i = (\hat{\phi}_x, \hat{\phi}_y)$. They give the pose $\boldsymbol{T}_i$ and the calibration $\boldsymbol{K}_i$ with focal lengths $f_x = W/(2\tan(\phi_x/2))$, $f_y = H/(2\tan(\phi_y/2))$ and the principal point at the image center.
- **Depth map** $\hat{\boldsymbol{D}}_i \in \mathbb{R}^{H\times W}$: the depth of every pixel.
- **Point map** $\hat{\boldsymbol{X}}_i \in \mathbb{R}^{H\times W\times 3}$: the 3D point of every pixel, in the frame of camera 1, which is the world frame.
- **Feature map** $\boldsymbol{F}_i \in \mathbb{R}^{H\times W\times C}$: C numbers per pixel, used to track points across the images.

The outputs are redundant, since the cameras follow from the point maps (PnP), yet predicting all of them improves accuracy.

Example 5.5.3: From field of view to focal length

For $W = 518$ pixels and a predicted horizontal field of view $\phi_x = 60°$: $f_x = 518/(2\tan 30°) = 518/1.1547 \approx 449$ pixels. With the principal point at $(259, 259)$ this fully specifies $\boldsymbol{K}$ (no calibration needed, in contrast to Section 5.1).

Intuition

Classical SfM needs $\boldsymbol{K}$ from calibration or self-calibration (and fails when it is wrong). VGGT simply *guesses* the field of view from the image content as a human would (a wide-angle shot looks different from a telephoto one). Predicting the same geometry in several forms (cameras, depth, point maps) gives the network several consistent "views" of one underlying 3D structure to learn from.

5.5.5 Architecture: Alternating Attention *(slides 60)*

- **Image tokens,** as $\boldsymbol{H}^a$ in DUSt3R: a vision transformer (DINO) cuts each image $\boldsymbol{I}_i$ into patches and turns each patch into a token, a feature vector.
- **Camera token:** a transformer returns one vector per input token. A 518×518 image gives $37 \times 37 = 1369$ patch tokens, hence 1369 outputs, one per patch, for the per-pixel maps. One extra learned token per image, the same for every image, yields one extra output: the camera head reads $\hat{\boldsymbol{g}}_i$ from it.
- **Alternating attention,** $L = 24$ times: *global attention* among the tokens of all images (as the cross-attention of DUSt3R but for N images), then *frame attention* among the tokens of one image.
- **Outputs:** the camera head turns each camera token into $\hat{\boldsymbol{g}}_i$; the DPT (Dense Prediction Transformer) head turns the image tokens into $\hat{\boldsymbol{D}}_i, \hat{\boldsymbol{X}}_i$ and $\boldsymbol{F}_i$.

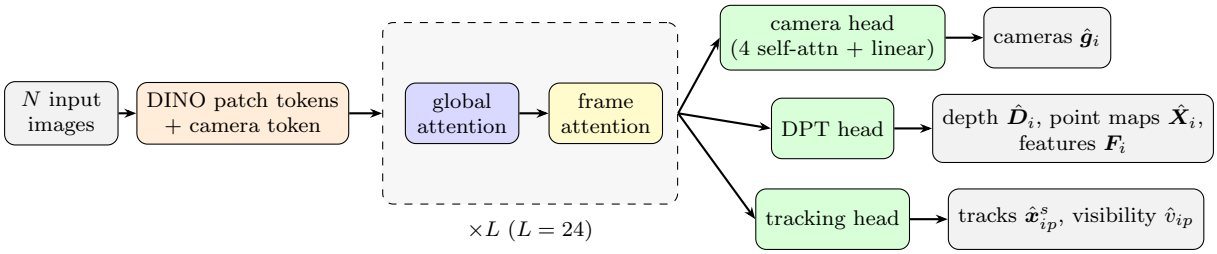


Figure 36: VGGT architecture (redrawn and simplified from slide 60).

Example 5.5.4: Cost of global versus frame attention

With $N = 10$ images and $T = 1369$ patch tokens per image there are $NT = 13{,}690$ tokens. Global attention scores $(NT)^2 \approx 1.87 \cdot 10^8$ token pairs per head and layer; frame attention scores $N \cdot T^2 \approx 1.87 \cdot 10^7$. Frame attention is $N = 10$ times cheaper.

Intuition

Global attention is where views "talk" to each other (it learns correspondence and relative pose implicitly); frame attention lets each image digest its own content cheaply. Alternating the two 24 times mimics what SfM does iteratively (match, then triangulate and refine), but inside one differentiable network with no hand-set thresholds. The camera token acts as a

learned "summary slot" per image from which the camera is read out; no temporal order is assumed, so the images may be an unordered photo collection.

5.5.6 Prediction Heads and Tracking *(slides 61)*

- **Camera head:** four self-attention layers and a linear layer turn the camera token of image i into $\hat{\boldsymbol{g}}_i$.
- **DPT head,** the Dense Prediction Transformer decoder (Ranftl et al., ICCV 2021), turns tokens back into per-pixel maps: $\hat{\boldsymbol{D}}_i$, $\hat{\boldsymbol{X}}_i$, $\boldsymbol{F}_i$, and confidence maps $\boldsymbol{C}_i^D, \boldsymbol{C}_i^X \in \mathbb{R}^{H\times W}$ that say how much to trust each pixel of $\hat{\boldsymbol{D}}_i$ and $\hat{\boldsymbol{X}}_i$.
- **Tracking head,** as in CoTracker2 (a point tracker for videos): given the pixel of a point p in one image, it compares the feature of that pixel with the feature maps $\boldsymbol{F}_i$ of all images, and predicts where the point appears in each image, $\hat{\boldsymbol{x}}_{ip}^s$, and whether it is visible there, $\hat{v}_{ip} \in [0,1]$.

5.5.7 Training Objective and Normalization *(slides 62)*

The four outputs are trained jointly, against ground truth written without a hat:

$$\mathcal{L} = \mathcal{L}_{\text{camera}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{pmap}} + \lambda\mathcal{L}_{\text{track}}, \qquad \lambda = 0.05,$$

$$\mathcal{L}_{\text{depth}} = \sum_{i=1}^{N} \left\|\boldsymbol{C}_i^D \odot (\hat{\boldsymbol{D}}_i - \boldsymbol{D}_i)\right\| + \left\|\boldsymbol{C}_i^D \odot (\nabla\hat{\boldsymbol{D}}_i - \nabla\boldsymbol{D}_i)\right\| - \alpha \log \boldsymbol{C}_i^D.$$

Here $\odot$ is the per-pixel product, ∇ the image gradient, and α the weight of the term $-\alpha \log \boldsymbol{C}_i^D$, which keeps the confidence from dropping to zero. $\mathcal{L}_{\text{camera}}$ is a Huber loss, squared for small errors and linear for large ones, between $\hat{\boldsymbol{g}}_i$ and $\boldsymbol{g}_i$; $\mathcal{L}_{\text{pmap}}$ is like $\mathcal{L}_{\text{depth}}$ for $\hat{\boldsymbol{X}}_i$ with $\boldsymbol{C}_i^X$; $\mathcal{L}_{\text{track}}$ is the distance of $\hat{\boldsymbol{x}}_{ip}^s$ to the true positions, plus a visibility term.

Intuition

The confidence $\boldsymbol{C}$ weights each pixel's error: unreliable pixels (sky, reflections, moving objects) can lower their confidence and so pay less. Without the $-\alpha \log \boldsymbol{C}$ term the network could cheat by predicting $\boldsymbol{C} = 0$ everywhere. The term charges a price for low confidence, so confidence drops only where errors would be large. The gradient term in the depth loss encourages sharp, correct edges.

Proposition 5.5.5: Why a normalization is required

Let $(\boldsymbol{T}_i, \boldsymbol{X})$ be a ground-truth reconstruction. Any similarity transform of the whole scene, applied consistently to cameras and points, produces the same images (Prop. 5.4.2). A regression target that is not fixed by a convention is therefore not a function of the images.

Proof. If two targets differ by a similarity, they map to identical inputs, so the network would be asked to output two different answers for the same input; the loss-minimizing prediction would blur between them. □

The images do not fix the global frame and scale (5.3). The ground truth is therefore expressed in the frame of camera 1 and divided by s, the mean distance of its 3D points to the origin; the network learns to predict in this normalized frame.

Intuition

It is the neural-network version of the gauge fixing of bundle adjustment: BA fixes the first pose and one distance; VGGT fixes the first pose and the average point distance ("scene scale = 1"). Every training scene is thus expressed in the same canonical units.

Training data and cost. 17 datasets; 64 A100 GPUs for nine days; 2 to 24 images per scene, at most 518 pixels per side (the slide quotes around 1.5M EUR for this compute).

5.5.8 Results: Camera Pose Estimation *(slides 63)*

10 random frames per scene; time on one H100 GPU. AUC@30: area under the accuracy curve of the relative rotation and translation errors, up to 30°. RealEstate10K: walkthrough videos of houses, used for training by none of the methods. CO3Dv2: videos around everyday objects. PixSfM: COLMAP with feature-metric refinement. DUSt3R and MASt3R with global alignment.

Method	RealEstate10K ↑	CO3Dv2 ↑	Time
COLMAP + SuperPoint + SuperGlue	45.2	25.3	$\approx 15\,\mathrm{s}$
PixSfM	49.4	30.1	$> 20\,\mathrm{s}$
DUSt3R	67.7	76.7	$\approx 7\,\mathrm{s}$
MASt3R	76.4	81.8	$\approx 9\,\mathrm{s}$
VGGSfM v2	78.9	83.4	$\approx 10\,\mathrm{s}$
VGGT, feed-forward	85.3	88.2	$\approx 0.2\,\mathrm{s}$
VGGT + bundle adjustment	**93.5**	**91.8**	$\approx 1.8\,\mathrm{s}$

Table 3: Camera pose estimation, AUC@30 (slide 63).

Intuition

Ten random frames is a sparse, wide-baseline setting, the weakest regime of classical SfM (few overlaps, little parallax per pair). That is where a learned prior helps most: COLMAP+SuperPoint+SuperGlue gets 45.2 on RealEstate10K, VGGT 85.3 in a fifth of a second. On dense captures with thousands of images, incremental SfM remains the reference (5.4). The speed difference (0.2 s versus $\sim 15\,\mathrm{s}$) is a factor of about 75.

5.5.9 Results: Dense Geometry *(slides 64)*

DTU (lecture 04), Chamfer distance in mm (lower is better); GeoMVSNet is a learned MVS method given the true cameras.

DTU			ETH3D (laser-scanned GT)	
Method	GT cameras	Overall↓	Method	Overall↓
GeoMVSNet	yes	0.295	DUSt3R	1.005
MASt3R	yes	0.374	MASt3R	0.826
DUSt3R	no	1.741	VGGT, point head	0.709
VGGT	no	0.382	VGGT, depth + camera	**0.677**

Table 4: Dense geometry (slide 64).

- Depth unprojected with the predicted cameras beats the point head: the decomposition into depth and camera pays off.

- VGGT obtains dense depth without known cameras, whereas COLMAP's MVS needs SfM first (5.4); on DTU, VGGT (no GT cameras) is within 0.01 mm of MASt3R, which *uses* the true cameras.

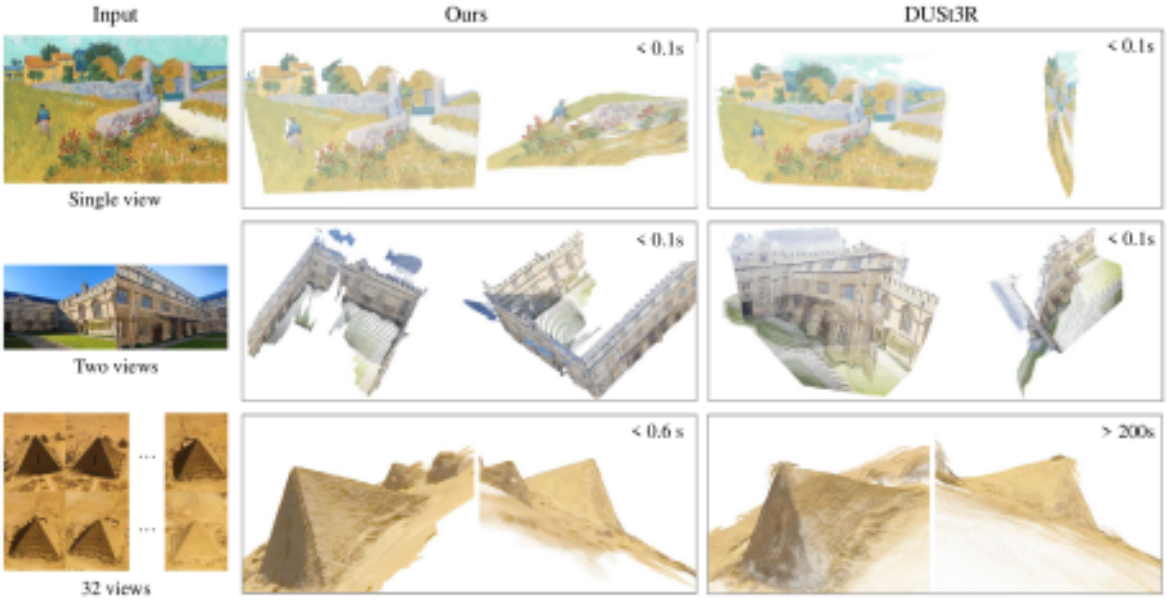


Figure 37: Input, VGGT ("Ours") and DUSt3R: one image, two images without overlap, 32 images.

5.5.10 Results: Matching and Tracking *(slides 65)*

ScanNet-1500: 1500 indoor image pairs; pose from the matches through the essential matrix (5.3); AUC of the pose error.

Method	AUC@5°	AUC@10°	AUC@20°
SuperGlue	16.2	33.8	51.8
LoFTR	22.1	40.8	57.6
RoMa	31.8	53.4	70.9
VGGT	**33.9**	**55.2**	**73.4**

Table 5: Two-view matching, pose from the essential matrix (slide 65).

RoMa is a dense learned matcher and the strongest baseline. The tracking head is *not trained* for two-view matching. The features also serve other tasks: a dynamic point tracker improves when given them, and novel view synthesis works without the input cameras.

Intuition

It is a surprising result: a network trained for 3D reconstruction produces features that outperform dedicated two-view matchers, because "being the same 3D point" is a stronger training signal than "looking alike". Note how the pipeline of Section 5.3 (matches $\rightarrow \tilde{\boldsymbol{E}} \rightarrow$ pose) is used here as an *evaluation protocol*, which shows the classical geometry remains the common language.



Figure 38: Point tracking: VGGT tracks (top) and a dynamic point tracker (CoTracker) given VGGT features (bottom) (Wang et al., CVPR 2025).

5.5.11 Bundle Adjustment from VGGT Predictions *(slides 66)*

Use the same objective as before,

$$\mathcal{T}^*, \mathcal{X}_w^* = \arg\min_{\mathcal{T},\mathcal{X}_w} \sum_{i=1}^{N} \sum_{p=1}^{P} w_{ip} \left\| \boldsymbol{x}_{ip}^s - \pi_i(\boldsymbol{x}_p^w) \right\|_2^2,$$

with the following ingredients, all predicted:

- observations $\boldsymbol{x}_{ip}^s$ and weights w_{ip}: tracks and visibilities from the tracking head;
- initial poses $\boldsymbol{T}_i$: the camera head, $\hat{\boldsymbol{g}}_i$, which also gives the calibrations $\boldsymbol{K}_i$;
- initial points $\mathcal{X}_w$: the predicted depth maps $\hat{\boldsymbol{D}}_i$, unprojected with the predicted cameras.

Initialization, image registration and triangulation (5.4) are no longer needed; with bundle adjustment the reconstruction takes about 2 s. AUC@30 on RealEstate10K rises from 85.3 to 93.5 with bundle adjustment.

Intuition

The two worlds complement each other: the network gives a very good starting point in the right basin of attraction (solving the non-convexity problem of Section 5.4), and BA then enforces what the network cannot guarantee, namely that the output is *exactly* geometrically consistent with the observed tracks. This is the same "closed-form initialization + non-linear refinement" pattern as in Zhang's calibration.

5.5.12 Limitations *(slides 67)*

Stated in the paper:

- no fisheye or panoramic images;
- accuracy drops under extreme rotations between the input images;
- minor non-rigid motion is handled, substantial non-rigid deformation fails;
- runtime and memory grow with the number of frames: 200 frames take 8.75 s and 40.63 GB of GPU memory (one H100, images of 336×518 pixels).

Beyond the paper:

- training needs large 3D-labeled datasets and large compute: 17 datasets, 64 A100 GPUs for nine days;
- no geometric guarantee: the output is a prediction, not the optimum of an objective;
- bundle adjustment still improves the result: 85.3 to 93.5 AUC@30 on RealEstate10K.

5.5.13 Comparison of Incremental and Feed-Forward SfM *(slides 68)*

	Incremental SfM	VGGT
Camera model	perspective, calibrated or self-calibrated	perspective, principal point at the center
Views	one at a time	all jointly
Assumptions	overlap and texture, a good initial pair	scenes like the training data, mostly rigid
Solution	RANSAC and nonlinear least squares	one forward pass of a transformer
Runtime	minutes to hours	under a second for tens of frames
Scale	thousands of images	hundreds of frames, bounded by GPU memory
Output	sparse points and cameras; dense with MVS	cameras, depth, point maps, tracks
Failure modes	low overlap, little texture, drift	fisheye images, extreme rotations, non-rigid motion
Guarantees	local optimum of the reprojection error	none

Table 6: Incremental SfM (Schönberger and Frahm, CVPR 2016) versus VGGT (Wang et al., CVPR 2025), slide 68.

Intuition

The two are complementary rather than competing: use the feed-forward network when views are few, wide-baseline or the time budget is tight (robots, interactive tools), and use classical incremental or global SfM when you have thousands of images and need a certified, repeatable optimum. The hybrid VGGT + BA gives the best accuracy.

Summary *(slides 69)*

Comparison of the methods covered

Method	Input → output	Core idea	Main limitation
Zhang calibration (5.1)	plane images → $\boldsymbol{K}$, poses	homography constraints on $\boldsymbol{B}$, then non-linear LS	needs a known target
SIFT (5.2)	image → 128D features	DoG extrema in scale space, gradient histograms	hand-crafted; slow
SuperPoint (5.2)	image → keypoints + 256D descriptors	self-supervised, homographic adaptation	fails for rotations not in training data
8-point / 5-point (5.3)	≥ 8 / 5 matches → $\tilde{\boldsymbol{E}}$	linear epipolar constraint (+ rank-2 projection)	scale unobservable; 4-fold ambiguity
Triangulation (5.3)	2 views → 3D point	DLT or reprojection error	ill-conditioned for small parallax
Bundle adjustment (5.4)	observations → poses, points	Gauss–Newton on SE(3) + Schur complement	needs good initialization
Incremental SfM / COLMAP (5.4)	image set → sparse model	seed from two views, register, triangulate, BA	slow; drift; many thresholds
VGGT (5.5)	N images → cameras, depth, points, tracks	alternating attention, trained on 17 datasets	no guarantees; GPU memory
VGGT + BA (5.5)	N images → refined model	learned initialization + BA	extra ~ 1.6 s

Table 7: Summary of the methods of Lecture 05.

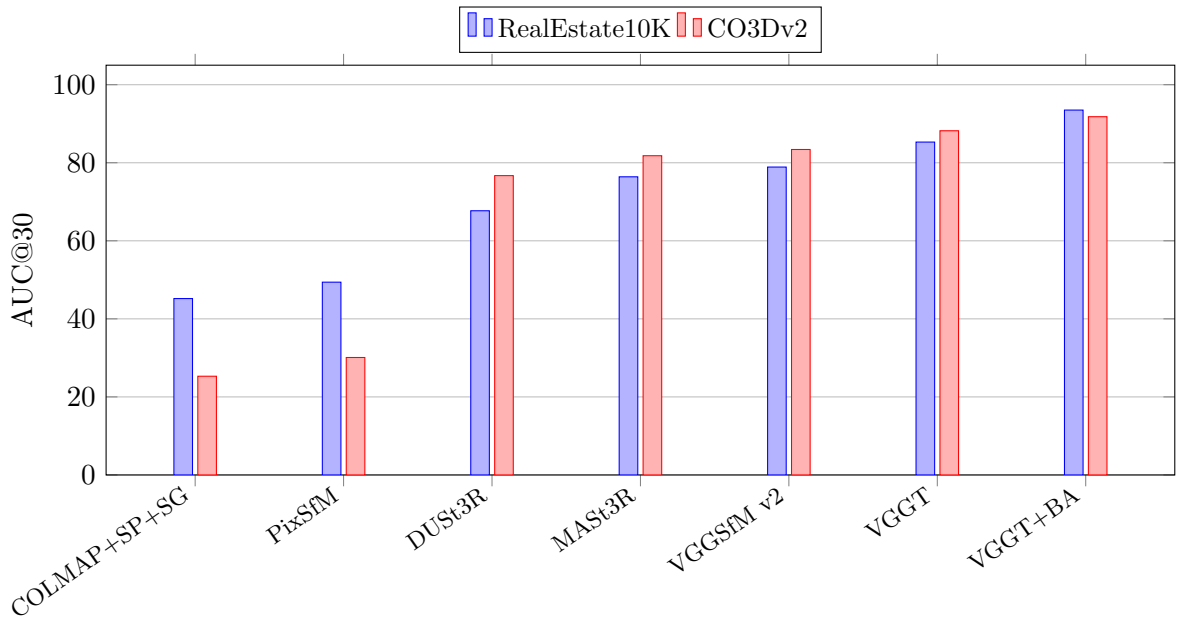


Figure 39: Camera pose accuracy of classical, learned two-view, and feed-forward methods (data of the table on slide 63; ten random frames per scene). The jump from the classical pipeline to learned methods is largest in the sparse regime, and bundle adjustment on top of VGGT adds the final gain.

Key take-aways

1. **Geometry supplies the constraints:** the epipolar constraint, triangulation and the reprojection error are the common language of every method in this lecture, classical or learned (even VGGT is evaluated and refined with them).
2. **Classical SfM = robust estimation + non-linear least squares.** Sparsity and the Schur complement keep bundle adjustment tractable (a factor $\sim 10^8$ in our example); its hard part is the initialization, which is why reconstruction is built incrementally from a good seed.
3. **Scale and gauge are never observable from images alone:** the essential matrix is defined up to scale and the four-fold decomposition ambiguity is resolved by cheirality; BA fixes one pose and one distance; VGGT normalizes its training targets.
4. **VGGT learns the whole mapping** from images to cameras, depth, point maps and tracks in one forward pass of a transformer with alternating global and frame attention.
5. **The most accurate poses come from a learned initialization refined by bundle adjustment** (AUC@30: 85.3 $\rightarrow$ 93.5 on RealEstate10K).

Acknowledgments (slide 70): the slides are adapted from the Computer Vision course at the University of Tübingen (Prof. Andreas Geiger); Section 5.5 figures and results are from Wang et al., "VGGT: Visual Geometry Grounded Transformer", CVPR 2025.